Lecture 4: CycleGAN, StyleGAN, VQ-GAN

비쌍 이미지 변환, 스타일 제어, 트랜스포머 기반 이미지 생성

목차

1. 개요
   - 1.1 핵심 개념
   - 1.2 선수 지식
   - 1.3 학습 목표
2. CycleGAN
   - 2.1 핵심 아이디어
   - 2.2 모델 구조
   - 2.3 손실 함수
   - 2.4 훈련 데이터
   - 2.5 생성 결과 및 한계
   - 2.6 PyTorch 구현
3. DiscoGAN
   - 3.1 CycleGAN과의 관계
   - 3.2 차이점
   - 3.3 결과 예시
4. StyleGAN
   - 4.1 ProGAN의 한계와 StyleGAN의 동기
   - 4.2 핵심 아이디어
   - 4.3 모델 구조
   - 4.4 스타일 믹싱 (Style Mixing)
   - 4.5 StyleGAN 시리즈 발전
   - 4.6 PyTorch 구현
5. VQ-GAN
   - 5.1 핵심 아이디어
   - 5.2 전체 파이프라인: 2-Stage Training
   - 5.3 Stage 1: VQ-GAN 상세
   - 5.4 코드북 인덱스 시퀀스의 이해
   - 5.5 Stage 2: Autoregressive Prior (트랜스포머 디코더)
   - 5.6 조건부 생성 (Conditional Generation)
   - 5.7 조건 부여 방식의 발전
   - 5.8 훈련/추론 과정 전체 정리
   - 5.9 PyTorch 구현
6. VQ-GAN에서 DALL-E로의 연결
   - 6.1 DALL-E (2021)
   - 6.2 DALL-E 2 (2022)
7. 핵심 수식 총정리
8. 참고문헌

---

1. 개요

이번 강의에서는 GAN의 응용 및 발전 모델들과, GAN을 넘어 **트랜스포머 기반 이미지 생성**으로 연결되는 VQ-GAN을 다룬다.

> 💡 **이 강의의 큰 그림: GAN의 세 가지 진화 방향** 기본 GAN(Lecture 3)은 "랜덤 노이즈 → 이미지"라는 단순한 파이프라인이었다. 그러나 실용적으로 쓰려면 세 가지 한계를 극복해야 한다:
>
> 1. **제어 불가능**: 어떤 이미지가 나올지 지정할 수 없다 → **StyleGAN**이 스타일 제어를 해결
> 2. **Paired data 필요**: 이미지 변환을 하려면 정확히 대응하는 쌍이 필요하다 → **CycleGAN**이 unpaired 변환을 해결
> 3. **CNN의 한계**: CNN 기반 생성은 장거리 의존성(long-range dependency)을 잘 포착하지 못한다 → **VQ-GAN**이 트랜스포머를 도입하여 해결
>
> 이 세 모델은 각각 독립적인 문제를 해결하면서, 동시에 **"이미지 생성의 질과 제어 가능성을 어떻게 높일 것인가"**라는 공통된 질문에 답한다.

1.1 핵심 개념

모델	연도	핵심
CycleGAN	2017.03	비쌍(unpaired) 이미지-이미지 변환
DiscoGAN	2017.03	CycleGAN과 동일 아이디어 (한국 연구, 15일 먼저 발표)
StyleGAN	2018	스타일 주입으로 이미지 속성 제어
VQ-GAN	2020	VQ-VAE + GAN + **트랜스포머 디코더로 이미지 생성**

1.2 선수 지식

- **확률론**: 조건부 확률, 기댓값, 분포 간 거리(KL divergence)
  - *왜 필요한가*: GAN의 손실 함수가 두 분포(실제 데이터 분포 vs 생성 분포) 사이의 거리를 최소화하는 것이므로, 분포 간 거리 개념이 핵심이다. Cycle Consistency Loss의 기댓값 표현 $\mathbb{E}_{x\sim p_{\text{data}}}[\cdot]$을 읽고 해석할 수 있어야 한다.
- **GAN 기초**: Minimax 게임, Adversarial Loss, Generator/Discriminator 구조 (Lecture 3 참조)
  - *왜 필요한가*: 이번 강의의 모든 모델이 Adversarial Loss를 기본으로 사용한다. Generator가 Discriminator를 "속이는" 게임 구조를 이해하지 못하면, 각 모델의 손실 함수 설계 의도를 파악할 수 없다.
- **VQ-VAE**: 코드북, 벡터 양자화, Straight-Through Estimator (Lecture 2 참조)
  - *왜 필요한가*: VQ-GAN은 VQ-VAE의 직접적인 확장이다. 코드북이 뭔지, 양자화가 뭔지 모르면 VQ-GAN의 Stage 1을 전혀 이해할 수 없다. 특히 Straight-Through Estimator (양자화의 불연속적 argmin 연산을 역전파할 수 있게 만드는 트릭)는 반드시 복습이 필요하다.
- **U-Net / ResNet**: Skip connection, Residual block의 개념
  - *왜 필요한가*: CycleGAN Generator가 ResNet 기반이고, Discriminator가 PatchGAN 구조이다. Skip connection이 왜 그래디언트 소실을 방지하는지 직관적으로 이해하고 있어야 한다.
- **트랜스포머**: Self-Attention, Cross-Attention, Causal Mask, 자기회귀적 생성
  - *왜 필요한가*: VQ-GAN의 Stage 2가 GPT와 동일한 자기회귀적 트랜스포머 디코더이다. Causal Mask가 왜 필요한지 (미래 토큰을 보면 치팅이므로), Cross-Attention이 조건부 생성에서 어떻게 작동하는지 이해해야 한다.

1.3 학습 목표

1. CycleGAN의 Cycle Consistency Loss가 비쌍 데이터에서 구도 보존을 가능하게 하는 원리 이해
2. StyleGAN의 Mapping Network와 AdaIN을 통한 스타일 분리(disentanglement) 메커니즘 이해

3. VQ-GAN의 2-Stage 파이프라인과 이미지 토큰화를 통한 트랜스포머 기반 이미지 생성 원리 습득
4. 조건부 생성(Conditional Generation)에서 토큰 연결 방식과 Cross-Attention 방식의 차이 비교
5. CycleGAN, StyleGAN, VQ-GAN 각각의 PyTorch 구현 능력 배양

---

2. CycleGAN

> 📄 **논문**: Zhu et al., "Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks", 2017.03 (UC Berkeley)
>
> 저자: Pix2Pix(2016) 논문의 저자와 동일

2.1 핵심 아이디어

> 💡 **왜 이런 접근이 필요한가?** 기존 GAN(예: DCGAN)은 랜덤 노이즈 z에서 이미지를 "무에서" 생성하는 방식이었다. 그러나 실제 응용에서 가장 수요가 많은 것은 **이미 있는 이미지를 다른 도메인의 이미지로 변환**하는 것이다 (사진을 그림처럼, 여름을 겨울처럼, 말을 얼룩말처럼). Pix2Pix(2016)가 이를 시도했지만, **정확히 같은 구도의 입출력 쌍(paired data)**이 필요했다. 현실에서 이런 쌍을 구하기 극히 어렵다 -- 예를 들어, 같은 포즈의 말과 얼룩말 사진은 존재하지 않는다. CycleGAN은 **cycle consistency**라는 제약 조건을 도입하여, 쌍이 없는 데이터만으로도 의미 있는 변환을 가능하게 한 것이 핵심 기여이다.

기존 GAN은 잠재 벡터 z를 입력으로 받아 이미지를 생성했다. CycleGAN은 다르다:

- **입력이 이미지, 출력도 이미지** (Image-to-Image Translation)
- 이미지의 **스타일을 변환**한다: 얼룩말 → 말, 여름 → 겨울, 사진 → 모네 화풍 등

직관적으로 말하면, CycleGAN은 "이 이미지의 **내용(content)**은 그대로 두고, **스타일(style)**만 바꿔라"라는 명령을 수행하는 모델이다. 여기서 "내용"이란 구도, 물체의 위치와 형태를 의미하고, "스타일"이란 색감, 질감, 시각적 특성을 의미한다.

Pix2Pix와의 차이

	Pix2Pix (2016)	**CycleGAN (2017)**
훈련 데이터	**Paired** (동일 구도의 쌍)	**Unpaired** (쌍 불필요)
예시	같은 구도의 여름/겨울 사진 쌍	여름 사진 1만장 + 겨울 사진 1만장 (별개)
데이터 수집	매우 어려움	쉬움

CycleGAN의 핵심 기여: **쌍이 없는(unpaired) 데이터만으로** 이미지 스타일 변환이 가능

> 💡 **Paired vs Unpaired가 왜 그렇게 중요한가?** Paired data란 "입력 이미지 A와 정확히 대응하는 출력 이미지 B"가 **한 쌍으로** 존재하는 것이다. 예를 들어, Pix2Pix에서 건물 외곽선(segmentation map)을 실제 건물 사진으로 변환하려면, **동일한 건물**의 외곽선과 사진이 쌍으로 필요하다. 이 경우는 외곽선을 알고리즘으로 추출할 수 있으니 쌍을 만들 수 있다. 그러나 "말 → 얼룩말" 변환이라면? 같은 포즈, 같은 배경, 같은 조명 조건의 말 사진과 얼룩말 사진이 필요한데, 이런 데이터는 **자연계에 존재하지 않는다**. CycleGAN은 이 근본적 제약을 깬 것이다 -- 그냥 말 사진 모음과 얼룩말 사진 모음을 각각 인터넷에서 수집하면 된다.

응용 예시

- 흑백 사진 → 컬러 사진
- 겨울 풍경 → 봄/여름/가을
- 고양이 → 개

- 사과 → 오렌지
- 사진 → 모네/고흐/세잔 화풍

> ⚠️ **주의**: 하나의 CycleGAN 모델이 이 모든 것을 수행하는 것이 아니다. 말↔얼룩말 모델, 여름↔겨울 모델 등 **각각 별도의 모델**을 훈련해야 한다.

2.2 모델 구조

> 💡 **왜 Generator가 2개, Discriminator가 2개 필요한가?** CycleGAN의 핵심 아이디어는 **양방향 변환**이다. 말→얼룩말 변환만 학습하면, "cycle"을 만들 수 없다. 얼룩말→말→얼룩말이라는 왕복 여행을 하려면 **반대 방향** 변환기도 있어야 한다. 따라서 $G_{X\to Y}$와 $G_{Y\to X}$ 두 개의 Generator가 필요하다. 각 Generator가 만든 이미지를 판별할 Discriminator도 각각 필요하므로, 총 4개의 네트워크가 훈련된다. 이것은 계산 비용이 2배라는 의미이지만, unpaired data로 의미 있는 변환을 얻기 위해 치러야 할 대가이다.

CycleGAN은 **4개의 네트워크**가 필요하다:

네트워크	역할	예시 (얼룩말↔말)
$G_{X\to Y}$	도메인 X → 도메인 Y 변환	얼룩말 → 말
$G_{Y\to X}$	도메인 Y → 도메인 X 변환	말 → 얼룩말
D_Y	Y 도메인 판별자	진짜 말인지 가짜 말인지 판별
D_X	X 도메인 판별자	진짜 얼룩말인지 가짜 얼룩말인지 판별

> 💡 **만약 Generator가 1개뿐이라면?** Generator $G_{X\to Y}$만 있다고 가정하자. 이 경우 "말→얼룩말→말"이라는 cycle을 구성할 수 없으므로 Cycle Consistency Loss를 계산할 수 없다. Adversarial Loss만으로 훈련하면, Generator는 "얼룩말 도메인의 분포에서 그럴싸한 이미지"를 출력하겠지만, 입력 말의 구도와 전혀 관련 없는 랜덤한 얼룩말을 출력할 수 있다. 이를 **mode collapse** (모드 붕괴 -- Generator가 입력에 무관하게 소수의 비슷한 출력만 반복 생성하는 현상)라 한다. 양방향 Generator와 Cycle Consistency가 이 문제를 방지한다.

Generator 구조: U-Net

기존 GAN의 Generator는 저차원 벡터 z를 받아 이미지를 생성했다 (디코더 형태). CycleGAN의 Generator는 **이미지를 입력으로 받아 이미지를 출력**하므로, 입출력 차원이 같다:

```
이미지 (128×128×3) → [Encoder ↓] → 압축 표현 → [Decoder ↑] → 이미지 (128×128×3)
```

이 구조가 U자 형태라서 **U-Net**이라고 부른다.

- 양쪽 차원이 같고 가운데가 홀쭉한 형태
- **Residual Connection(Skip Connection)** (인코더의 각 층 출력을 디코더의 대응 층 입력에 직접 연결하는 구조 -- 인코더에서 잃어버릴 수 있는 고해상도 세부 정보를 디코더로 직접 전달): 그래디언트 소실 방지, 세부 정보 보존
- 생성 모델에서 매우 널리 사용 (Diffusion Model, Flow Matching 등에서도 U-Net 사용)

> 💡 **왜 Skip Connection이 이미지 변환에서 특히 중요한가?** 이미지 변환(말→얼룩말)에서 변하는 것은 **질감과 색상** 정도이고, 물체의 형태와 공간 배치는 그대로 유지해야 한다. 인코더가 이미지를 저차원으로 압축하면 전체적인 의미(semantic)는 보존되지만, **디테일한 공간 정보**(엣지의 정확한 위치, 미세한 질감 패턴)는 손실된다. Skip Connection이 인코더의 고해상도 피처를 디코더에 직접 전달함으로써, 디코더는 "무엇을 생성할지"는 bottleneck에서 결정하고, "어디에 어떤 디테일을 넣을지"는 Skip Connection에서 가져오는 분업이 가능해진다.

```
x ──→ [Conv↓] ──→ [Conv↓] ──→ [Conv↓] ──→ [Conv↑] ──→ [Conv↑] ──→ [Conv↑] ──→ ŷ
        │                                                                  ↑
        └──────────────── Skip Connection ──────────────────────────────┘
```

> ⚠️ **참고**: 실제 CycleGAN 논문에서는 U-Net 대신 **ResNet 기반 Generator** (9개 또는 6개의 Residual Block)를 사용한다. U-Net은 Pix2Pix에서 사용된 구조이며, CycleGAN에서도 옵션으로 제공된다. ResNet 기반을 선택한 이유는, CycleGAN에서는 입력과 출력의 구조가 매우 유사하므로(말과 얼룩말은 형태가 거의 같고 질감만 다르다), Residual Block이 "입력과의 차이(residual)"만 학습하면 되어 효율적이기 때문이다.

Discriminator 구조: PatchGAN

CycleGAN은 **PatchGAN Discriminator** (이미지 전체에 대해 하나의 진짜/가짜 판정을 내리는 대신, 이미지를 겹치는 패치들로 나누어 각 패치별로 독립적인 진짜/가짜 판정을 내리는 구조)를 사용한다 (Pix2Pix에서 제안됨).

기존 Discriminator vs PatchGAN:

	기존 Discriminator	**PatchGAN**
입력	전체 이미지	이미지를 **패치 단위**로 분할
출력	스칼라 1개 (진짜/가짜)	**패치별 진짜/가짜** (예: 16개 판정)
비유	심판 1명이 전체 경기 판정	**구역별 심판**이 각자 판정

예를 들어 이미지를 $4 \times 4 = 16$개 패치로 나누면, 각 패치에 대해 독립적으로 진짜/가짜 판정을 하고 결과를 평균 또는 다수결로 최종 판정한다.

> 💡 **왜 PatchGAN이 전체 이미지를 보는 Discriminator보다 나은가?** 이미지 변환에서 가장 중요한 것은 **질감(texture)의 사실성**이다. 전체 이미지를 한 번에 보는 Discriminator는 "전반적으로 말처럼 보이는가?"에 초점을 맞추지만, **국소적인 질감 패턴**(말 털의 결, 피부의 광택, 배경의 잔디 질감)이 사실적인지는 잘 파악하지 못한다. PatchGAN은 각 70x70 패치가 "이 부분만 놓고 보면 진짜 말처럼 보이는가?"를 판정하므로, 모든 영역에서 질감이 균일하게 사실적이 되도록 강제한다. 또한, 더 작은 receptive field를 사용하므로 **파라미터 수가 적고 훈련이 빠르다**는 실용적 이점도 있다.

PatchGAN의 장점:

- **로컬 구조(local structure)**를 더 잘 평가
- 작은 패치만 다루므로 **파라미터 수가 적음** (경량 모델)
- 전체 이미지를 한 번에 보는 것보다 **세부 질감을 잘 판별**

2.3 손실 함수

CycleGAN의 손실은 크게 **두 가지**로 구성된다. 각각이 해결하는 문제를 명확히 구분하자:

- **Adversarial Loss**: "출력이 목표 도메인의 이미지**처럼 보이게**" 만드는 역할 (품질)
- **Cycle Consistency Loss**: "입력의 **구도와 내용을 유지**"하게 만드는 역할 (일관성)

이 두 손실이 균형을 이루어야 좋은 결과가 나온다.

(1) Adversarial Loss (적대적 손실)

> 💡 **Adversarial Loss의 역할과 한계** Adversarial Loss (생성자와 판별자가 서로 경쟁하는 minimax 게임에서 유도되는 손실)는 "생성된 이미지가 목표 도메인의 실제 이미지와 구별 불가능하게" 만드는 역할을 한다. 직관적으로 말하면, D_Y라는 말 전문가가 "이건 진짜 말인가 가짜 말인가?"를 판정하고,

> $G_{X\to Y}$는 이 전문가를 속이도록 훈련된다. 문제는, 전문가를 속이는 방법이 여러 가지라는 것이다 -- 입력 얼룩말의 구도를 무시하고 **완전히 다른 말 이미지**를 출력해도, 그것이 사실적이기만 하면 Adversarial Loss는 만족한다. 이것이 바로 Adversarial Loss**만으로는** 부족한 이유이다.

기존 GAN과 동일한 형태의 손실. 방향별로 각각 존재한다.

$X \to Y$ **방향** (얼룩말 → 말):

$$\mathcal{L}_{\text{GAN}}(G_{X\to Y}, D_Y) = \mathbb{E}_{y\sim p_{\text{data}}(y)}[\log D_Y(y)] + \mathbb{E}_{x\sim p_{\text{data}}(x)}[\log(1 - D_Y(G_{X\to Y}(x)))]$$

이 수식을 단계별로 분해하면:

- **첫 번째 항** $\mathbb{E}_y[\log D_Y(y)]$: 진짜 말 이미지 y를 판별자에 넣었을 때, $D_Y(y)$가 1에 가까워야 $\log(1) = 0$으로 손실이 최소화된다. 즉, "진짜를 진짜로 판별"하도록 D_Y를 훈련한다.
- **두 번째 항** $\mathbb{E}_x[\log(1 - D_Y(G_{X\to Y}(x)))]$: 얼룩말 x를 Generator가 변환한 가짜 말 $G_{X\to Y}(x)$를 판별자에 넣었을 때, D_Y**의 관점**에서는 $D_Y(G_{X\to Y}(x))$가 0에 가까워야 $\log(1-0) = 0$으로 손실이 최소화된다 (가짜를 가짜로 판별). 반면 G**의 관점**에서는 $D_Y(G_{X\to Y}(x))$가 1에 가까워야 한다 (판별자를 속여야 하므로).
- 이것이 **minimax 게임**: D_Y는 이 손실을 최대화하려 하고, $G_{X\to Y}$는 최소화하려 한다.
- $D_Y(y)$: 진짜 말 사진에 대한 판별 → 1에 가까워야 함
- $D_Y(G_{X\to Y}(x))$: 얼룩말을 변환한 가짜 말에 대한 판별 → 0에 가까워야 함 (판별자 관점)

$Y \to X$ **방향** (말 → 얼룩말):

$$\mathcal{L}_{\text{GAN}}(G_{Y\to X}, D_X) = \mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D_X(x)] + \mathbb{E}_{y\sim p_{\text{data}}(y)}[\log(1 - D_X(G_{Y\to X}(y)))]$$

위 수식은 $X \to Y$와 완전히 대칭적이다. 방향만 반대일 뿐, 구조는 동일하다: $G_{Y\to X}$가 말을 얼룩말로 변환하고, D_X가 그것이 진짜 얼룩말인지 판별한다.

Adversarial Loss만으로는 "진짜 말처럼 보이는 이미지"는 만들 수 있지만, **원래 구도를 유지하는 보장이 없다**. 마주보고 있는 얼룩말을 넣어도 전혀 다른 구도의 말이 나올 수 있다. 직관적으로, Adversarial Loss는 "목적지에 도착했는가?"만 확인하지 "어떤 경로로 왔는가?"는 확인하지 않는 것과 같다.

(2) Cycle Consistency Loss (순환 일관성 손실)

> 💡 **왜 Cycle Consistency라는 아이디어가 탄생했는가?** Unpaired data에서 Generator를 훈련하면, 입력-출력 사이의 **대응 관계가 전혀 정의되지 않는다**. 얼룩말 사진 1만장과 말 사진 1만장이 있을 때, "이 얼룩말은 이 말에 대응한다"라는 정보가 없다. Adversarial Loss만으로는 Generator가 임의의 mapping을 학습할 수 있다 -- 예를 들어, 모든 얼룩말 입력에 대해 동일한 말 이미지를 출력하는 것도 Adversarial Loss를 줄일 수 있다(mode collapse). Cycle Consistency는 이 문제를 해결하는 **자기 감독 (self-supervision)** 신호이다: "변환한 것을 다시 되돌렸을 때 원본이 복원되어야 한다"는 제약이, 의미 있는 1:1 대응을 강제한다.

핵심 아이디어: 얼룩말 → 말 → 다시 얼룩말로 변환하면, **원래 이미지로 돌아와야 한다**.

$$\mathcal{L}_{\text{cyc}}(G_{X\to Y}, G_{Y\to X}) = \mathbb{E}_{x\sim p_{\text{data}}(x)}\left[\|G_{Y\to X}(G_{X\to Y}(x)) - x\|_1\right] + \mathbb{E}_{y\sim p_{\text{data}}(y)}\left[\|G_{X\to Y}(G_{Y\to X}(y)) - y\|_1\right]$$

이 수식을 단계별로 분해하면:

- $\|\cdot\|_1$**은 L1 norm** (각 픽셀 차이의 절대값 합): L2 norm($\|\cdot\|_2^2$, 차이의 제곱 합)보다 **블러링이 적은** 복원을 유도한다. L2는 큰 오류에 과도한 벌칙을 주어 "안전한 평균값"으로 수렴하는 경향이 있는 반면, L1은 모든 오류에 균등한 벌칙을 주어 더 선명한 결과를 만든다.
- $\mathbb{E}_{x\sim p_{\text{data}}(x)}[\cdot]$**는 기댓값**: 훈련 데이터의 모든 이미지에 대해 평균적으로 복원이 잘 되어야 한다는 의미이다. 실제 구현에서는 미니배치 평균으로 근사한다.

각 항을 분석하면:

Forward Cycle ($X \to Y \to X$):

$$x \xrightarrow{G_{X\to Y}} \hat{y} \xrightarrow{G_{Y\to X}} \hat{x} \approx x$$

- 얼룩말 x → 말 $\hat{y}$ → 다시 얼룩말 $\hat{x}$
- $\hat{x}$와 x가 **픽셀 레벨에서** 일치해야 함 (L1 norm)

Backward Cycle ($Y \to X \to Y$):

$$y \xrightarrow{G_{Y\to X}} \hat{x} \xrightarrow{G_{X\to Y}} \hat{y} \approx y$$

- 말 y → 얼룩말 $\hat{x}$ → 다시 말 $\hat{y}$
- $\hat{y}$와 y가 픽셀 레벨에서 일치해야 함

> **왜 구도가 유지되는가?** 얼룩말을 말로 바꾼 뒤 다시 얼룩말로 되돌려야 하므로, 중간 단계인 "말"이 원래 얼룩말의 구도 정보를 보존해야만 한다. 오토인코더처럼 동작하여 구도를 유지시킨다.

> 💡 **Cycle Consistency의 심층 이해: 정보 보존 원리** 수학적으로 생각하면, $G_{Y\to X}(G_{X\to Y}(x)) = x$가 성립하려면, $G_{X\to Y}$의 출력 $\hat{y}$에 원본 x를 **완벽하게 복원할 수 있는 충분한 정보**가 담겨 있어야 한다. 만약 $G_{X\to Y}$가 변환 과정에서 구도 정보를 버렸다면 (예: 모든 얼룩말을 같은 포즈의 말로 변환), $G_{Y\to X}$가 아무리 뛰어나도 원래 구도를 복원할 수 없다. 따라서 Cycle Consistency Loss를 최소화하려면, **두 Generator 모두 입력의 구조적 정보를 보존해야만 한다**. 이것이 unpaired 설정에서 내용(content) 보존이 자연스럽게 달성되는 원리이다.
>
> 비유하자면: 영어를 한국어로 번역한 뒤 다시 영어로 번역했을 때 원래 문장이 나오려면, 한국어 번역이 원래 의미를 충실히 담고 있어야 한다. 의미를 왜곡하거나 누락하면 역번역이 불가능하다.

(3) 전체 손실 함수

$$\boxed{\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{GAN}}(G_{X\to Y}, D_Y) + \mathcal{L}_{\text{GAN}}(G_{Y\to X}, D_X) + \lambda \cdot \mathcal{L}_{\text{cyc}}(G_{X\to Y}, G_{Y\to X})}$$

각 항의 역할을 정리하면:

- $\mathcal{L}_{\text{GAN}}(G_{X\to Y}, D_Y)$: $G_{X\to Y}$가 만든 이미지가 Y 도메인(말)의 실제 분포와 구별 불가능하도록 만든다. → "사실적인 말처럼 보여야 한다"
- $\mathcal{L}_{\text{GAN}}(G_{Y\to X}, D_X)$: $G_{Y\to X}$가 만든 이미지가 X 도메인(얼룩말)의 실제 분포와 구별 불가능하도록 만든다. → "사실적인 얼룩말처럼 보여야 한다"
- $\lambda \cdot \mathcal{L}_{\text{cyc}}$: 양방향 변환의 왕복이 원본을 복원하도록 한다. → "구도와 내용을 보존해야 한다"
- λ: 하이퍼파라미터 — Cycle Consistency의 비중 조절
  - λ 크면: 구도 유지 강조 (변환 품질 약간 저하 가능). 직관적으로, Generator가 "안전하게" 최소한의 변환만 수행하려 한다.
  - λ 작으면: 변환 품질 강조 (구도 유지 약화 가능). Generator가 더 자유롭게 변환하지만, 구도가 무너질 위험이 있다.
  - $\lambda = 0$이면: Cycle Consistency가 없으므로 unpaired 변환의 의미가 사라진다. 그냥 두 개의 독립적인 GAN을 훈련하는 것과 같다.

하이퍼파라미터 λ 설정 팁: 각 손실의 값 범위를 관찰해야 한다. Adversarial Loss가 10~20 범위이고 Cycle Consistency Loss가 0.1~0.2 범위라면, λ를 크게 설정해야 두 손실의 균형이 맞는다.

> ⚠️ **실전 팁**: 논문에서는 $\lambda = 10$을 사용한다. 훈련 초기에 각 손실의 크기를 로깅하여 균형을 확인하는 것이 좋다. WandB나 TensorBoard로 $\mathcal{L}_{\text{GAN}}$과 $\mathcal{L}_{\text{cyc}}$를 각각 추적하면서, 한쪽이 지나치게 지배적이지 않은지 모니터링해야 한다.

(4) Identity Loss (부가 손실)

> 💡 **왜 Identity Loss가 필요한가?** $G_{X\to Y}$는 "입력을 Y 도메인으로 변환"하는 역할이다. 그런데 **이미 Y 도메인인 이미지**를 넣으면? 변환이 필요 없으므로 입력을 그대로 출력해야 한다. Identity Loss 없이는 Generator가 이미 올바른 도메인의 이미지도 불필요하게 변형할 수 있다. 예를 들어 색상 변환(풍경의 색조를 바꾸는 작업)에서, Identity Loss 없이는 Generator가 이미 올바른 색조의 이미지까지 색을 왜곡시킬 수 있다. Identity Loss는 **"이미 목표 도메인에 있으면 건드리지 마라"**는 정규화(regularization) 역할을 한다.

$G_{X\to Y}$에 이미 Y 도메인인 이미지를 넣으면 변환이 일어나지 않아야 한다:

$$\mathcal{L}_{\text{identity}} = \mathbb{E}_y[\|G_{X\to Y}(y) - y\|_1] + \mathbb{E}_x[\|G_{Y\to X}(x) - x\|_1]$$

각 항의 의미:

- $\|G_{X\to Y}(y) - y\|_1$: "말→말 변환" Generator에 진짜 말을 넣으면, 출력도 입력과 동일해야 한다. 즉 $G_{X\to Y}$는 Y 도메인에 대해 **항등 함수(identity function)**처럼 동작해야 한다.
- $\|G_{Y\to X}(x) - x\|_1$: 마찬가지로, $G_{Y\to X}$에 진짜 얼룩말을 넣으면 변화가 없어야 한다.

오류 방지를 위한 보조적 손실로, 핵심은 아니지만 색상 보존이 중요한 작업에서는 효과적이다. 논문에서는 특히 Monet 화풍 변환 등에서 Identity Loss가 색상 안정성을 크게 개선함을 보였다.

2.4 훈련 데이터

- 페어링 불필요: 얼룩말 사진 ~1만장 + 말 사진 ~1만장 (별개)
- 구도가 맞을 필요 없음

2.5 생성 결과 및 한계

성공 사례

- 풍경 사진 → 모네/고흐/세잔 화풍 변환
- 여름 ↔ 겨울 풍경 변환
- 말 ↔ 얼룩말 변환 (구도 유지)

이 성공 사례들의 공통점은 **두 도메인 사이의 구조적 차이가 작다**는 것이다. 말과 얼룩말은 체형이 거의 같고 줄무늬 패턴만 다르며, 여름과 겨울 풍경은 색감과 질감만 다르다. 이처럼 "질감/색상 수준의 변환"에서 CycleGAN이 강력하다.

실패 사례

- **형태가 다른 객체 변환**: 사과 → 오렌지 — 색은 바뀌지만 **사과 모양이 유지됨** (Cycle Consistency가 픽셀 레벨 일치를 요구하므로)
- **구조적 차이가 큰 변환**: 고양이 → 개 — 털 질감은 바뀌지만 **얼굴 형태가 고양이**
- **배경 인물 변환**: 말 → 얼룩말 시 말 위에 탄 사람도 같이 줄무늬가 됨

> 💡 **실패의 근본 원인: Cycle Consistency의 양날의 검** Cycle Consistency Loss는 L1 norm으로 **픽셀 레벨 일치**를 요구한다. 이것이 구도 보존의 장점이지만, 동시에 **형태 변환을 불가능**하게 만드는 제약이기도 하다. 사과를 오렌지로 바꾸려면 형태(둥근 정도, 꼭지 모양)도 바뀌어야 하는데, 형태를 바꾸면 역변환(오렌지→사과)으로 원래 사과 모양을 복원할 수 없다. 따라서 Generator는 "형태는 건드리지 않고 색만 바꾸는" 안전한 전략을 택한다. 이것은 CycleGAN의 본질적 한계이며, 이후 CUT(Contrastive Unpaired Translation, 2020) 등의 연구에서 패치 레벨 대조 학습으로 이 문제를 완화하려는 시도가 이어졌다.

2.6 PyTorch 구현

Residual Block (CycleGAN Generator의 핵심 빌딩 블록)

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class ResidualBlock(nn.Module):
    """
    CycleGAN Generator에 사용되는 Residual Block.
    입력과 출력의 채널 수가 동일하며, Instance Normalization을 사용한다.
    """
    def __init__(self, in_channels):
        super(ResidualBlock, self).__init__()

        self.block = nn.Sequential(
            nn.ReflectionPad2d(1),                    # 패딩 (반사 패딩으로 경계 아티
            nn.Conv2d(in_channels, in_channels, 3),   # 3×3 conv, same channels
            nn.InstanceNorm2d(in_channels),           # Instance Normalization (배
            nn.ReLU(inplace=True),
            nn.ReflectionPad2d(1),
            nn.Conv2d(in_channels, in_channels, 3),
            nn.InstanceNorm2d(in_channels),
        )

    def forward(self, x):
        # Skip connection: 입력을 그대로 더함 (ResNet 방식)
        return x + self.block(x)


class CycleGANGenerator(nn.Module):
    """
    CycleGAN의 ResNet 기반 Generator.
    구조: 다운샘플링 → 9개 Residual Block → 업샘플링

    Args:
        in_channels: 입력 이미지 채널 수 (RGB=3)
        out_channels: 출력 이미지 채널 수 (RGB=3)
        n_residual_blocks: Residual Block 수 (논문 기본값: 9)
    """
    def __init__(self, in_channels=3, out_channels=3, n_residual_blocks=9):
        super(CycleGANGenerator, self).__init__()

        # ---- 초기 Convolution (7×7) ----
        model = [
            nn.ReflectionPad2d(3),
            nn.Conv2d(in_channels, 64, 7),
            nn.InstanceNorm2d(64),
            nn.ReLU(inplace=True),
        ]

        # ---- 다운샘플링 (2번, 해상도 1/4로 축소) ----
        in_features = 64
        out_features = in_features * 2
        for _ in range(2):
            model += [
                nn.Conv2d(in_features, out_features, 3, stride=2, padding=1),
                nn.InstanceNorm2d(out_features),
                nn.ReLU(inplace=True),
            ]
            in_features = out_features
            out_features = in_features * 2
```

```python
        # ---- Residual Blocks (9개) ----
        for _ in range(n_residual_blocks):
            model += [ResidualBlock(in_features)]

        # ---- 업샘플링 (2번, 해상도 4배로 복원) ----
        out_features = in_features // 2
        for _ in range(2):
            model += [
                nn.ConvTranspose2d(in_features, out_features, 3,
                                   stride=2, padding=1, output_padding=1),
                nn.InstanceNorm2d(out_features),
                nn.ReLU(inplace=True),
            ]
            in_features = out_features
            out_features = in_features // 2

        # ---- 출력 Convolution (7×7) ----
        model += [
            nn.ReflectionPad2d(3),
            nn.Conv2d(64, out_channels, 7),
            nn.Tanh(),  # 출력을 [-1, 1] 범위로 제한
        ]

        self.model = nn.Sequential(*model)

    def forward(self, x):
        """
        Args:
            x: 입력 이미지 (B, C, H, W), 도메인 X의 이미지
        Returns:
            변환된 이미지 (B, C, H, W), 도메인 Y의 이미지
        """
        return self.model(x)
```

PatchGAN Discriminator

```python
class PatchGANDiscriminator(nn.Module):
    """
    PatchGAN Discriminator (70×70 receptive field).
    전체 이미지가 아닌 패치 단위로 진짜/가짜를 판별한다.
    출력은 스칼라가 아니라 (B, 1, H', W') 형태의 판별 맵이다.

    Args:
        in_channels: 입력 이미지 채널 수 (RGB=3)
    """
    def __init__(self, in_channels=3):
        super(PatchGANDiscriminator, self).__init__()

        def discriminator_block(in_filters, out_filters, normalize=True):
            """하나의 Discriminator 레이어: Conv → [InstanceNorm] → LeakyReLU"""
            layers = [nn.Conv2d(in_filters, out_filters, 4, stride=2, padding=1)]
            if normalize:
                layers.append(nn.InstanceNorm2d(out_filters))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers

        self.model = nn.Sequential(
            # 첫 레이어는 normalization 없이
            *discriminator_block(in_channels, 64, normalize=False),  # (B, 64, H/2
            *discriminator_block(64, 128),                            # (B, 128, H
            *discriminator_block(128, 256),                           # (B, 256, H
            *discriminator_block(256, 512),                           # (B, 512, H
            nn.ZeroPad2d((1, 0, 1, 0)),                              # 비대칭 패딩
            nn.Conv2d(512, 1, 4, padding=1),                         # (B, 1, H/16
            # Sigmoid를 적용하지 않음 → LSGAN loss 사용 시
        )

    def forward(self, img):
        """
        Args:
            img: 입력 이미지 (B, C, H, W)
        Returns:
            패치별 판별 결과 (B, 1, H', W') — 각 값이 해당 패치의 진짜/가짜 점수
        """
        return self.model(img)
```

CycleGAN 손실 함수 및 훈련 루프

```python
import itertools


class CycleGANLoss:
    """
    CycleGAN의 세 가지 손실 함수를 관리하는 클래스.
    - Adversarial Loss (LSGAN 방식)
    - Cycle Consistency Loss (L1)
    - Identity Loss (L1, 선택적)
    """
    def __init__(self, lambda_cyc=10.0, lambda_identity=5.0):
        self.lambda_cyc = lambda_cyc          # Cycle Loss 가중치 (논문 기본값: 10)
        self.lambda_identity = lambda_identity # Identity Loss 가중치
        self.criterion_GAN = nn.MSELoss()     # LSGAN: BCE 대신 MSE 사용
        self.criterion_cycle = nn.L1Loss()    # Cycle Consistency: L1 loss
        self.criterion_identity = nn.L1Loss() # Identity: L1 loss

    def adversarial_loss(self, pred, is_real):
        """
        LSGAN adversarial loss.
        진짜면 target=1, 가짜면 target=0인 MSE loss.
        """
        target = torch.ones_like(pred) if is_real else torch.zeros_like(pred)
        return self.criterion_GAN(pred, target)

    def cycle_consistency_loss(self, real_img, reconstructed_img):
        """
        Cycle Consistency Loss: ||G_YX(G_XY(x)) - x||_1
        원본과 재구성 이미지의 L1 거리.
        """
        return self.criterion_cycle(reconstructed_img, real_img) * self.lambda_cyc

    def identity_loss(self, real_img, same_img):
        """
        Identity Loss: ||G_XY(y) - y||_1
        이미 목표 도메인인 이미지를 넣으면 변하지 않아야 함.
        """
        return self.criterion_identity(same_img, real_img) * self.lambda_identity


def train_cyclegan(G_XY, G_YX, D_X, D_Y, dataloader_X, dataloader_Y,
                   num_epochs, device, lr=0.0002):
    """
    CycleGAN 전체 훈련 루프.

    Args:
        G_XY: X→Y Generator
        G_YX: Y→X Generator
        D_X: X 도메인 Discriminator
        D_Y: Y 도메인 Discriminator
        dataloader_X: 도메인 X 이미지 DataLoader
        dataloader_Y: 도메인 Y 이미지 DataLoader
        num_epochs: 에폭 수
        device: 'cuda' 또는 'cpu'
        lr: 학습률
    """
    # Generator와 Discriminator를 별도의 옵티마이저로 훈련
    optimizer_G = torch.optim.Adam(
        itertools.chain(G_XY.parameters(), G_YX.parameters()),
        lr=lr, betas=(0.5, 0.999)
    )
```

```
optimizer_D_X = torch.optim.Adam(D_X.parameters(), lr=lr, betas=(0.5, 0.999))
optimizer_D_Y = torch.optim.Adam(D_Y.parameters(), lr=lr, betas=(0.5, 0.999))

loss_fn = CycleGANLoss(lambda_cyc=10.0, lambda_identity=5.0)

for epoch in range(num_epochs):
    for real_X, real_Y in zip(dataloader_X, dataloader_Y):
        real_X = real_X[0].to(device)  # (B, 3, H, W)
        real_Y = real_Y[0].to(device)

        # ========================
        # (1) Generator 훈련
        # ========================
        optimizer_G.zero_grad()

        # Identity Loss: G_XY(y) ≈ y, G_YX(x) ≈ x
        identity_Y = G_XY(real_Y)
        loss_id_Y = loss_fn.identity_loss(real_Y, identity_Y)
        identity_X = G_YX(real_X)
        loss_id_X = loss_fn.identity_loss(real_X, identity_X)

        # Adversarial Loss: G_XY(x)가 진짜처럼 판별되도록
        fake_Y = G_XY(real_X)               # 얼룩말 → 가짜 말
        pred_fake_Y = D_Y(fake_Y)
        loss_GAN_XY = loss_fn.adversarial_loss(pred_fake_Y, is_real=True)

        fake_X = G_YX(real_Y)               # 말 → 가짜 얼룩말
        pred_fake_X = D_X(fake_X)
        loss_GAN_YX = loss_fn.adversarial_loss(pred_fake_X, is_real=True)

        # Cycle Consistency Loss
        reconstructed_X = G_YX(fake_Y)      # 가짜 말 → 재구성 얼룩말
        loss_cyc_X = loss_fn.cycle_consistency_loss(real_X, reconstructed_X)

        reconstructed_Y = G_XY(fake_X)      # 가짜 얼룩말 → 재구성 말
        loss_cyc_Y = loss_fn.cycle_consistency_loss(real_Y, reconstructed_Y)

        # 전체 Generator Loss
        loss_G = (loss_GAN_XY + loss_GAN_YX
                  + loss_cyc_X + loss_cyc_Y
                  + loss_id_X + loss_id_Y)
        loss_G.backward()
        optimizer_G.step()

        # ========================
        # (2) Discriminator D_Y 훈련
        # ========================
        optimizer_D_Y.zero_grad()

        loss_real_Y = loss_fn.adversarial_loss(D_Y(real_Y), is_real=True)
        loss_fake_Y = loss_fn.adversarial_loss(D_Y(fake_Y.detach()), is_real=F
        loss_D_Y = (loss_real_Y + loss_fake_Y) * 0.5

        loss_D_Y.backward()
        optimizer_D_Y.step()

        # ========================
        # (3) Discriminator D_X 훈련
        # ========================
        optimizer_D_X.zero_grad()
```

```
        loss_real_X = loss_fn.adversarial_loss(D_X(real_X), is_real=True)
        loss_fake_X = loss_fn.adversarial_loss(D_X(fake_X.detach()), is_real=F
        loss_D_X = (loss_real_X + loss_fake_X) * 0.5

        loss_D_X.backward()
        optimizer_D_X.step()

    print(f"[Epoch {epoch+1}/{num_epochs}] "
          f"G: {loss_G.item():.4f}  D_X: {loss_D_X.item():.4f}  D_Y: {loss_D_Y
```

---

3. DiscoGAN

> 📄 **논문**: Kim et al., "Learning to Discover Cross-Domain Relations with Generative Adversarial Networks", 2017.03 (한국)
>
> CycleGAN보다 **15일 먼저** 발표 (arXiv 기준)

3.1 CycleGAN과의 관계

DiscoGAN은 CycleGAN과 **완전히 동일한 아이디어**:

- Generator 2개 (A→B, B→A)
- Discriminator 2개 (A 판별, B 판별)
- **Cycle Consistency Loss** (입력을 변환한 뒤 다시 역변환하면 원본과 동일해야 한다는 제약 조건)로 구도 유지

우연히 동시에 독립적으로 발표된 것이다.

> 💡 **과학에서의 동시 발견 현상** DiscoGAN과 CycleGAN의 동시 발표는 과학사에서 반복되는 패턴이다 (뉴턴과 라이프니츠의 미적분, 다윈과 월레스의 진화론 등). 이는 "시대가 무르익으면 같은 아이디어가 독립적으로 나타난다"는 것을 보여준다. 2017년 당시 GAN 연구자들 사이에서 unpaired translation이 핵심 과제였고, cycle consistency라는 아이디어는 자연스러운 해결책이었기에 두 팀이 독립적으로 동일한 결론에 도달한 것이다.

3.2 차이점

- DiscoGAN은 **저해상도** 이미지(64x64)로 실험 → CycleGAN(256x256)에 비해 주목도 낮음
- DiscoGAN은 기본 CNN 구조를 사용한 반면, CycleGAN은 ResNet 기반 Generator와 PatchGAN Discriminator라는 더 강력한 아키텍처를 채택하여 고해상도에서도 잘 동작
- 그럼에도 15일 먼저 발표되었으므로 우선권이 있음

3.3 결과 예시

- 여자 → 남자 얼굴 변환
- 금발 → 흑발 변환
- 자동차 → 사람 얼굴 변환 (방향성 유지: 왼쪽 보는 차 → 왼쪽 보는 사람)
- 스케치 가방 → 컬러 가방
- 스케치 신발 → 컬러 신발

특히 "자동차 → 사람 얼굴" 변환이 흥미로운데, 자동차가 왼쪽을 향하면 사람도 왼쪽을 향하고, 자동차가 빨간색이면 사람의 배경이나 피부톤이 따뜻한 색으로 나온다. 이는 Cycle Consistency가 **의미적 대응(semantic correspondence)**을 자동으로 학습한다는 강력한 증거이다.

---

4. StyleGAN

> 📄 **논문**: Tero Karras et al., "A Style-Based Generator Architecture for GANs", 2018 (NVIDIA)
>
> ProGAN의 저자 Tero Karras가 개발. 매년 새 버전을 발표하며 지속적으로 발전.

4.1 ProGAN의 한계와 StyleGAN의 동기

> 💡 **왜 ProGAN만으로는 부족한가?** ProGAN(Progressive GAN)은 4x4 해상도에서 시작하여 점진적으로 1024x1024까지 키우는 **Progressive Growing** (저해상도에서 시작하여 훈련 중 점진적으로 해상도를 높여가는 훈련 전략) 방식으로 고해상도 얼굴 이미지를 성공적으로 합성했다. 그러나 ProGAN을 비롯한 기존 GAN들은 모두 동일한 근본적 문제를 공유한다: **잠재 벡터 z가 이미지의 모든 속성을 한꺼번에 결정**한다는 것이다. z의 한 차원을 바꾸면 나이, 성별, 포즈, 배경이 **동시에** 변할 수 있다. 이를 **entangled representation**(얽힌 표현)이라 한다. 사용자가 "이 얼굴에서 나이만 바꾸고 나머지는 유지해주세요"라고 요청하면, 기존 GAN으로는 이를 수행할 방법이 없다.

ProGAN은 고해상도 이미지를 성공적으로 합성했지만, 잠재 벡터 z를 넣으면 **어떤 이미지가 나올지 제어할 수 없었다**:

- 수염 달린 남자를 만들고 싶어도 직접 지정 불가
- 원하는 스타일의 이미지가 나오는 z를 역으로 찾아야 함
- 스타일 핸들링이 불가능

StyleGAN의 목표: **스타일을 제어 가능**하게 만들기. 구체적으로는, 이미지의 **고수준 속성(성별, 나이, 포즈)**과 **저수준 속성(피부 질감, 머리카락 색, 배경 색)**을 **독립적으로** 제어할 수 있게 만드는 것이다.

4.2 핵심 아이디어

기존 GAN: $z \to G \to$ 이미지 (z가 직접 이미지 생성에 관여)

> 💡 **기존 방식의 문제를 구체적으로 이해하기** 기존 GAN에서 z는 Generator의 첫 번째 레이어에 **한 번만** 입력된다. Generator의 내부 레이어들이 이 z를 가공하여 이미지를 만드는데, 이 과정에서 z의 어떤 차원이 이미지의 어떤 속성에 대응하는지가 자연스럽게 **분리(disentangle)되지 않는다**. z 공간에서 가까운 두 벡터가 시각적으로 유사한 이미지를 만들리라는 보장도 없다. StyleGAN은 이 문제를 z를 **직접 사용하지 않고** 학습된 변환을 거친 w를 사용함으로써 해결한다.

StyleGAN의 접근:

1. 이미지 생성의 **시작점을 고정** (학습 가능한 상수 텐서 $4 \times 4 \times 512$)
2. 잠재 벡터 z는 **스타일 제어용**으로만 사용
3. 스타일을 이미지 생성 과정의 **중간중간에 주입(injection)**

직관적으로 말하면, 기존 GAN이 "씨앗(z)을 심으면 나무(이미지)가 자란다"는 방식이었다면, StyleGAN은 "기본 나무 틀(constant)은 이미 있고, 물감(style code)을 단계별로 칠해 원하는 나무를 만든다"는 방식이다.

4.3 모델 구조

(1) 고정 시작점 (Learned Constant)

$$\text{Constant Tensor: } c \in \mathbb{R}^{4\times4\times512}$$

- 훈련 중에는 학습되어 값이 변하지만, 훈련 완료 후 **고정**
- Positional Embedding과 유사한 개념: 훈련 중 학습, 이후 고정
- 이 텐서는 **평균적인 이미지를 생성하기 위한 기본값**

> 💡 **왜 고정 시작점을 사용하는가?** 기존 GAN에서는 z가 Generator의 입력이었다. 즉 z가 "이미지의 뼈대"를 결정하고, 이후 레이어들이 살을 붙여 나갔다. 이 방식의 문제는, z의 역할이 너무 크다는 것이다 -- z가 이미지의 전체 구조를 결정하므로, z를 조금만 바꿔도 이미지가 크게 변한다. StyleGAN은 이 역할을 z에서 분리한다: **뼈대(constant)**는 고정시키고, z에서 유도된 스타일 코드가 "색칠"과 "변형"만 담당한다. 이렇게 하면 스타일 조절이 훨씬 부드럽고 예측 가능해진다.

(2) Mapping Network ($z \to w$)

$$z \sim \mathcal{N}(0, I) \xrightarrow{\text{Normalize}} \bar{z} \xrightarrow{f_\theta} w$$

- 잠재 벡터 z를 정규화한 뒤, **8개의 Fully Connected Layer**를 통과
- 출력 w는 z보다 **표현이 풍부한** 스타일 코드
- 왜 8개? 잘 분리(disentangle)된 스타일 공간을 얻기 위해 경험적으로 결정

> ⚠️ **Disentanglement가 중요한 이유**: z 공간에서는 "나이"와 "성별" 같은 속성이 얽혀 있을 수 있다. Mapping Network를 통과한 w 공간에서는 이들이 독립적으로 제어 가능해진다. 이를 **disentangled representation**이라 한다.

> 💡 **Mapping Network가 disentanglement를 달성하는 원리** z는 **표준 정규분포** $\mathcal{N}(0, I)$에서 샘플링된다. 이 분포는 구형(spherical)이므로, 실제 데이터의 속성 분포와 맞지 않을 수 있다. 예를 들어, 실제 얼굴 데이터에서 "수염이 있는 여성"은 매우 드물다 -- 즉 "성별"과 "수염" 속성은 강한 상관관계가 있다. z 공간의 균일한 구형 분포로는 이 상관 구조를 자연스럽게 표현할 수 없다.
>
> Mapping Network f_θ는 8개의 FC 레이어를 통해 z를 **비선형 변환**하여 w 공간으로 매핑한다. 이 w 공간은 정규분포의 제약에서 벗어나, **데이터의 실제 속성 분포에 맞는 형태**로 자유롭게 변형된다. 논문에서는 이를 정량적으로 검증하기 위해 **Perceptual Path Length**와 **Linear Separability** 메트릭을 사용하여, w 공간이 z 공간보다 더 disentangle되어 있음을 보였다.
>
> 직관적으로 말하면: z는 "정해진 형식의 설문지"이고, w는 "자유형식 자기소개서"이다. 설문지는 형식이 고정되어 있어 다양한 사람을 표현하기 어렵지만, 자기소개서는 형식 제약 없이 각자의 특성을 자유롭게 기술할 수 있다.

(3) Affine Transform ($w \to y$)

$$y = Aw + b$$

- 스타일 코드 w를 **어파인 변환** (선형 변환 Aw에 편향 b를 더하는 연산 — 회전, 스케일링, 이동을 포함하는 일반적 선형 변환)하여 **스케일 y_s와 바이어스 y_b**를 생성
- 층마다 다른 어파인 변환 행렬 A를 사용 — 이것이 핵심이다: **같은 w도 각 해상도 층에서 다른 방식으로 변환된다**. 저해상도 층의 어파인 변환은 w에서 전체적인 구조(포즈, 얼굴형)에 관련된 정보를 추출하고, 고해상도 층의 어파인 변환은 세부 디테일(피부 질감, 모발 색)에 관련된 정보를 추출한다.
- 보통 2~3개의 스타일 코드(y_1, y_2)를 사용

(4) AdaIN (Adaptive Instance Normalization)

> 💡 **왜 AdaIN이 스타일 주입에 효과적인가?** AdaIN은 원래 **Neural Style Transfer** (Huang & Belongie, 2017) 분야에서 제안된 기법이다. 핵심 관찰은 CNN 피처 맵의 **통계량(평균과 분산)이 이미지의 "스타일"을 인코딩**한다는 것이다. 예를 들어, 모네 그림의 피처 맵은 특정한 평균/분산 패턴을 가지고, 사진의 피처 맵은 다른 패턴을 가진다. AdaIN은 피처 맵의 통계량을 **외부에서 지정한 값으로 교체**함으로써, 콘텐츠는 유지하면서 스타일만 바꿀 수 있다. StyleGAN은 이 원리를 역이용하여, 스타일 코드 w로부터 원하는 통계량(y_s, y_b)을 생성하고 이를 피처 맵에 적용한다.

스타일을 이미지 생성 과정에 주입하는 핵심 메커니즘:

$$\text{AdaIN}(x, y) = y_s \cdot \frac{x - \mu(x)}{\sigma(x)} + y_b$$

단계별 풀이:

Step 1: 피처 맵 x를 정규화 (**Instance Normalization** -- Batch Normalization과 달리 배치 내 다른 샘플과 무관하게, **각 샘플의 각 채널을 독립적으로** 정규화한다)

$$\hat{x} = \frac{x - \mu(x)}{\sigma(x)}$$

$\mu(x)$와 $\sigma(x)$는 각 피처 맵의 평균과 표준편차. 고등학교 통계의 표준화(z-score) 공식과 동일하다. 이 과정을 거치면 피처 맵의 원래 통계량(즉, 원래의 스타일 정보)이 **완전히 제거**되어 평균 0, 분산 1이 된다.

Step 2: 스타일로 스케일링 + 이동

$$\text{output} = y_s \cdot \hat{x} + y_b$$

- y_s: 스케일 팩터 (피처 맵의 **분산을 결정** → 콘트라스트, 질감의 강도를 제어)
- y_b: 바이어스 (피처 맵의 **평균을 결정** → 밝기, 색조를 제어)

직관적으로 말하면: Step 1에서 "기존 스타일을 지우고"(정규화), Step 2에서 "새로운 스타일을 입힌다"(스케일링+이동). 이것이 style transfer의 본질이다.

> **해석**: 이미지가 그냥 CNN을 통과하며 생성되면 밋밋한 결과만 나온다. AdaIN이 중간중간에 피처 맵을 변형(스케일 조절 + 이동)시켜 **스타일을 주입**한다. Batch Normalization에서 학습 가능한 γ, β와 유사한 역할이지만, 여기서는 **샘플링된 w로부터** 결정된다.

> 💡 **BN vs IN vs AdaIN 비교로 이해 깊히기**
>
> - **Batch Normalization**: 배치 전체의 통계량으로 정규화 → 고정된 학습 가능 γ, β로 복원. 분류 등 일반 태스크에 적합.
> - **Instance Normalization**: 각 이미지 개별 통계량으로 정규화 → 고정된 학습 가능 γ, β로 복원. Style Transfer에서 "이미지별 스타일 제거"에 유용.
> - **AdaIN**: 각 이미지 개별 통계량으로 정규화 → **외부 입력(w)에서 생성된 동적 y_s, y_b**로 복원. StyleGAN의 핵심: 스타일을 **매번 다르게** 주입할 수 있다.

(5) Noise Injection

각 층에 **랜덤 노이즈**를 추가:

$$x' = x + B \cdot n, \quad n \sim \mathcal{N}(0, I)$$

- B: 학습 가능한 채널별 스케일링 가중치 (어떤 채널에 노이즈를 더 많이 줄지 학습)
- n: 공간적으로 다른 값을 가지지만 모든 채널에 동일한 노이즈 맵
- 같은 스타일 코드를 넣어도 노이즈가 다르면 **약간씩 다른 이미지** 생성
- 머리카락 배치, 피부 질감 등 **확률적 세부사항(stochastic detail)**을 담당

> 💡 **왜 스타일과 노이즈를 분리하는가?** 이것은 StyleGAN의 설계 철학에서 매우 중요한 부분이다. 이미지에는 두 종류의 변동(variation)이 있다:
>
> 1. **결정적 변동(deterministic variation)**: 성별, 나이, 포즈 등 → 스타일 코드 w가 담당
> 2. **확률적 변동(stochastic variation)**: 머리카락의 정확한 배치, 피부의 미세 질감, 배경의 랜덤 패턴 → 노이즈 n이 담당
>
> 만약 노이즈 없이 스타일 코드만 사용하면, 동일한 w에서 항상 **정확히 같은** 이미지가 나온다 (머리카락 한 올의 위치까지). 이는 비현실적이다. 노이즈를 추가하면 "같은 사람이지만 조금씩 다른 사진"을 생성할 수 있다.

(6) 전체 흐름

```
z ~ N(0,I) → [Normalize] → [FC ×8: Mapping Network] → w
                                                      |
                  ┌──── [Affine A₁] → y₁ (scale, bias) ←┘
                  │                                   |
                  │      ┌── [Affine A₂] → y₂ ←──────┘
                  │      │
Constant ──→ [Conv] → [AdaIN(y₁)] → [Conv] → [AdaIN(y₂)] → ··· → 이미지
  4×4×512     + noise               + noise
              ↑                     ↑
        [ProGAN 방식: 점진적 해상도 증가 4×4 → 8×8 → ··· → 1024×1024]
```

4.4 스타일 믹싱 (Style Mixing)

> 💡 **왜 스타일 믹싱이 가능한가?** StyleGAN의 구조에서 **각 해상도 층은 독립적인 스타일 코드를 받는다**. 저해상도 층(4x4, 8x8)은 전체적인 구조(포즈, 얼굴형)를 결정하고, 고해상도 층(256x256, 512x512)은 세부 디테일(피부 질감, 머리 색)을 결정한다. 이 구조 덕분에, **서로 다른 w에서 유도된 스타일 코드를 층별로 선택적으로 적용**할 수 있다. 이것은 기존 GAN에서는 불가능했던 것이다 -- 기존 GAN에서 z는 첫 번째 층에만 입력되므로, "전반부는 A 사람의 특성, 후반부는 B 사람의 특성"이라는 조합이 구조적으로 불가능했다.

다른 w에서 나온 스타일 코드를 **층별로 섞어** 사용하면, 이미지 속성을 조합할 수 있다.

두 종류의 스타일 코드

- y_1 **(Coarse style)**: 초반부(저해상도 층, 4x4~8x8)에 주입 → **전체적인 큰 틀** 제어 (얼굴형, 포즈, 이마 형태, 안경 유무 등)
- y_2 **(Fine style)**: 후반부(고해상도 층, 64x64~1024x1024)에 주입 → **미세한 부분** 제어 (머리 색, 피부 질감, 입술 색 등)

이 분리가 자연스러운 이유: CNN에서 **저해상도 피처 맵은 글로벌한 구조를 인코딩**하고 (넓은 receptive field), **고해상도 피처 맵은 로컬한 디테일을 인코딩**한다 (좁은 receptive field). 따라서 저해상도 층의 스타일은 글로벌 속성을, 고해상도 층의 스타일은 로컬 속성을 자연스럽게 제어한다.

스타일 믹싱 예시

소스 A의 w_A에서 y_1 추출, 소스 B의 w_B에서 y_2 추출하여 결합:

구성	결과
y_1: A의 코드, y_2: B의 코드	A의 얼굴형 + B의 색상/질감
y_1: 어린아이 코드, y_2: 성인 코드	어린아이의 전체적 형태 + 성인의 디테일

활용: 원하는 형태의 이미지를 만들었던 **코드를 기록**해두고, 나중에 조합하여 원하는 속성의 이미지를 생성. 이것은 일종의 **이미지의 벡터 연산**이다 -- "A의 얼굴 구조 + B의 색상"처럼 속성을 벡터 레벨에서 조합하는 것이 가능해진다.

4.5 StyleGAN 시리즈 발전

버전	연도	핵심 변화
StyleGAN v1	2018	ProGAN 구조 + AdaIN 스타일 주입
StyleGAN v2	2019	AdaIN 제거, Progressive Growing 제거, 더 자연스러운 생성

StyleGAN v3	2020	Aliasing 현상 해결 (DSP 기법 적용), 비디오 생성 가능

StyleGAN v1의 문제: 스타일을 조금 바꾸면 수염과 얼굴이 **따로 노는** 현상 (aliasing). StyleGAN v3에서 디지털 신호 처리(DSP) 기법을 적용하여 해결.

> 💡 **각 버전의 핵심 변화를 깊이 이해하기**
>
> - **v1** → **v2**: AdaIN이 "물방울 아티팩트(water droplet artifact)"를 만드는 문제가 발견되었다. AdaIN의 Instance Normalization 과정에서 피처 맵의 통계량이 급격하게 바뀌면서, 일부 공간 위치에 비정상적인 값이 집중되는 현상이다. v2에서는 AdaIN을 **weight demodulation** (컨볼루션 가중치 자체를 스타일에 맞게 조절하는 방식)으로 대체하여 이 문제를 해결했다. 또한 Progressive Growing도 제거하고 처음부터 전체 해상도에서 훈련하는 방식으로 전환했다.
> - **v2** → **v3**: 잠재 벡터를 연속적으로 변화시키면 (예: 얼굴을 천천히 회전), 피처가 **특정 픽셀 위치에 고정**(texture sticking)되어 부자연스러운 결과가 나오는 문제가 발견되었다. 이는 이미지를 이산적 픽셀 그리드에서 처리할 때 발생하는 aliasing과 동일한 현상이다. v3에서는 Shannon-Nyquist 표본화 정리 등 디지털 신호 처리 이론을 적용하여 모든 레이어에서 alias-free한 연산을 보장했다.

> 특징: 각 버전이 이전 방법을 **과감하게 버리고** 완전히 새로운 접근을 도입. 이는 연구 관점에서 매우 인상적인 태도이다 -- 자신의 이전 방법론에 집착하지 않고, 문제가 발견되면 근본적인 해결책을 찾는다.

4.6 PyTorch 구현

Mapping Network

```python
class MappingNetwork(nn.Module):
    """
    StyleGAN의 Mapping Network: z → w
    8개의 FC Layer를 통해 잠재 벡터 z를 스타일 공간 w로 매핑한다.
    w 공간은 z 공간보다 disentangle되어 있어 스타일 제어가 용이하다.

    Args:
        z_dim: 잠재 벡터 z의 차원 (기본 512)
        w_dim: 스타일 코드 w의 차원 (기본 512)
        num_layers: FC layer 수 (논문 기본값: 8)
    """
    def __init__(self, z_dim=512, w_dim=512, num_layers=8):
        super(MappingNetwork, self).__init__()

        layers = []
        for i in range(num_layers):
            in_dim = z_dim if i == 0 else w_dim
            layers.append(nn.Linear(in_dim, w_dim))
            layers.append(nn.LeakyReLU(0.2, inplace=True))

        self.mapping = nn.Sequential(*layers)

    def forward(self, z):
        """
        Args:
            z: 잠재 벡터 (B, z_dim), 표준 정규분포에서 샘플링
        Returns:
            w: 스타일 코드 (B, w_dim)
        """
        # PixelNorm (각 벡터를 L2 정규화)
        z = z / (torch.sqrt(torch.mean(z ** 2, dim=1, keepdim=True)) + 1e-8)
        w = self.mapping(z)
        return w
```

AdaIN (Adaptive Instance Normalization) 및 Noise Injection

```python
class AdaIN(nn.Module):
    """
    Adaptive Instance Normalization.
    피처 맵 x를 Instance Normalization한 뒤,
    스타일 코드 w에서 생성된 scale(γ)과 shift(β)를 적용한다.

    수식: AdaIN(x, y) = y_s * (x - μ(x)) / σ(x) + y_b

    Args:
        in_channels: 피처 맵의 채널 수
        w_dim: 스타일 코드 w의 차원
    """
    def __init__(self, in_channels, w_dim=512):
        super(AdaIN, self).__init__()

        # Instance Normalization (affine=False: 학습 가능한 파라미터 없이)
        self.instance_norm = nn.InstanceNorm2d(in_channels, affine=False)

        # 어파인 변환: w → (scale, bias), 각각 채널 수만큼
        self.style_transform = nn.Linear(w_dim, in_channels * 2)

    def forward(self, x, w):
        """
        Args:
            x: 피처 맵 (B, C, H, W)
            w: 스타일 코드 (B, w_dim)
        Returns:
            스타일이 주입된 피처 맵 (B, C, H, W)
        """
        # Step 1: Instance Normalization
        normalized = self.instance_norm(x)

        # Step 2: 스타일 코드에서 scale, bias 생성
        style = self.style_transform(w)             # (B, 2*C)
        gamma, beta = style.chunk(2, dim=1)         # 각각 (B, C)
        gamma = gamma.unsqueeze(2).unsqueeze(3)     # (B, C, 1, 1)
        beta = beta.unsqueeze(2).unsqueeze(3)       # (B, C, 1, 1)

        # Step 3: 스타일 적용 (scale + shift)
        return gamma * normalized + beta


class NoiseInjection(nn.Module):
    """
    StyleGAN의 Noise Injection.
    각 피처 맵에 학습 가능한 가중치로 스케일링된 노이즈를 추가한다.
    머리카락 배치, 피부 질감 등 확률적 세부사항을 담당한다.

    수식: x' = x + B * n,  n ~ N(0, I)
    """
    def __init__(self, in_channels):
        super(NoiseInjection, self).__init__()
        # 채널별 학습 가능한 노이즈 가중치
        self.weight = nn.Parameter(torch.zeros(1, in_channels, 1, 1))

    def forward(self, x):
        """
        Args:
            x: 피처 맵 (B, C, H, W)
        Returns:
            노이즈가 추가된 피처 맵 (B, C, H, W)
```

```python
        """
        B, C, H, W = x.shape
        noise = torch.randn(B, 1, H, W, device=x.device)  # 공간별 동일 노이즈 (채널
        return x + self.weight * noise


class StyleGANSynthesisBlock(nn.Module):
    """
    StyleGAN Synthesis Network의 하나의 블록.
    Conv → Noise Injection → AdaIN → LeakyReLU의 구조이다.

    Args:
        in_channels: 입력 채널 수
        out_channels: 출력 채널 수
        w_dim: 스타일 코드 차원
        upsample: 업샘플링 여부
    """
    def __init__(self, in_channels, out_channels, w_dim=512, upsample=True):
        super(StyleGANSynthesisBlock, self).__init__()

        self.upsample = nn.Upsample(scale_factor=2, mode='bilinear',
                                    align_corners=False) if upsample else None
        self.conv = nn.Conv2d(in_channels, out_channels, 3, padding=1)
        self.noise = NoiseInjection(out_channels)
        self.adain = AdaIN(out_channels, w_dim)
        self.activation = nn.LeakyReLU(0.2, inplace=True)

    def forward(self, x, w):
        """
        Args:
            x: 피처 맵 (B, C_in, H, W)
            w: 스타일 코드 (B, w_dim)
        Returns:
            출력 피처 맵 (B, C_out, 2H, 2W) 또는 (B, C_out, H, W)
        """
        if self.upsample is not None:
            x = self.upsample(x)       # 해상도 2배 증가
        x = self.conv(x)               # Convolution
        x = self.noise(x)              # Noise Injection
        x = self.adain(x, w)           # AdaIN으로 스타일 주입
        x = self.activation(x)         # LeakyReLU
        return x
```

간단한 StyleGAN Generator 예시

```python
class SimpleStyleGANGenerator(nn.Module):
    """
    StyleGAN Generator의 간략화된 버전.
    Mapping Network + Synthesis Network로 구성된다.

    실제 StyleGAN은 Progressive Growing 등 더 복잡한 기법을 사용하지만,
    여기서는 핵심 아이디어(Mapping → AdaIN 스타일 주입)를 보여주기 위한 간소화 구현이다.

    Args:
        z_dim: 잠재 벡터 차원
        w_dim: 스타일 코드 차원
        img_channels: 출력 이미지 채널 수
    """
    def __init__(self, z_dim=512, w_dim=512, img_channels=3):
        super(SimpleStyleGANGenerator, self).__init__()

        self.mapping = MappingNetwork(z_dim, w_dim, num_layers=8)

        # 학습 가능한 상수 (4×4×512) — StyleGAN의 고정 시작점
        self.constant = nn.Parameter(torch.randn(1, 512, 4, 4))

        # Synthesis blocks: 4×4 → 8×8 → 16×16 → 32×32 → 64×64
        self.blocks = nn.ModuleList([
            StyleGANSynthesisBlock(512, 512, w_dim, upsample=False),  # 4×4 → 4×4
            StyleGANSynthesisBlock(512, 256, w_dim, upsample=True),   # 4×4 → 8×8
            StyleGANSynthesisBlock(256, 128, w_dim, upsample=True),   # 8×8 → 16×1
            StyleGANSynthesisBlock(128, 64, w_dim, upsample=True),    # 16×16 → 32
            StyleGANSynthesisBlock(64, 32, w_dim, upsample=True),     # 32×32 → 64
        ])

        # 최종 출력 convolution
        self.to_rgb = nn.Conv2d(32, img_channels, 1)  # 1×1 conv → RGB

    def forward(self, z, style_mixing=False, mixing_layer=3):
        """
        Args:
            z: 잠재 벡터 (B, z_dim) 또는 (2, B, z_dim) for style mixing
            style_mixing: 스타일 믹싱 사용 여부
            mixing_layer: 스타일을 전환하는 레이어 인덱스
        Returns:
            생성 이미지 (B, img_channels, 64, 64)
        """
        if style_mixing:
            # 두 개의 서로 다른 z에서 w 생성
            w1 = self.mapping(z[0])   # Coarse style (전체 형태)
            w2 = self.mapping(z[1])   # Fine style (세부 질감)
        else:
            w1 = self.mapping(z)
            w2 = w1

        # 학습된 상수에서 시작
        x = self.constant.expand(z.shape[0] if not style_mixing else z.shape[1], -

        # 각 블록을 통과하며 스타일 주입
        for i, block in enumerate(self.blocks):
            # 지정된 레이어 이전에는 w1, 이후에는 w2 사용
            w = w1 if i < mixing_layer else w2
            x = block(x, w)

        # RGB 이미지로 변환
        img = self.to_rgb(x)
```

```
        return torch.tanh(img)
```

---

5. VQ-GAN

> 📄 **논문**: Esser, Rombach, Ommer, "Taming Transformers for High-Resolution Image Synthesis", 2020
>
> 논문 제목: "트랜스포머 길들이기" — 트랜스포머를 이미지 생성에 활용하는 것이 핵심
>
> 저자들은 이후 **Stable Diffusion** 모델의 저자이기도 하다.

5.1 핵심 아이디어

> 💡 **왜 VQ-GAN이 등장했는가? -- CNN 기반 생성의 한계** CycleGAN과 StyleGAN은 모두 CNN 기반 Generator를 사용한다. CNN은 **로컬한 패턴**(질감, 엣지)을 잘 포착하지만, **이미지 전체에 걸친 장거리 의존성(long-range dependency)**을 모델링하기 어렵다. 예를 들어, 얼굴의 왼쪽 눈과 오른쪽 눈이 대칭이어야 한다는 관계, 또는 건물의 창문이 일정한 패턴으로 반복되어야 한다는 관계는 CNN의 제한된 receptive field로는 포착하기 어렵다. 반면 **트랜스포머**는 Self-Attention을 통해 시퀀스의 어떤 위치든 직접 참조할 수 있어 장거리 의존성에 강하다. 문제는, 트랜스포머는 **이산적 토큰 시퀀스**를 다루도록 설계되었는데, 이미지는 **연속값 픽셀**로 이루어져 있다는 것이다. VQ-GAN은 VQ-VAE의 벡터 양자화를 사용하여 이미지를 이산적 토큰으로 변환함으로써, 트랜스포머를 이미지 생성에 적용할 수 있는 다리를 놓는다.

VQ-GAN의 본질:

1. **VQ-VAE + GAN**: 이미지를 이산적(discrete) 코드북 인덱스로 압축 (GAN 손실 추가로 복원 품질 향상)
2. **트랜스포머 디코더**: 이산화된 이미지 토큰을 **자기회귀적으로 생성** (GPT가 단어를 생성하듯)

> GAN 부분보다 **트랜스포머를 이미지 생성에 길들이는 것**이 핵심 기여. 논문 제목 "Taming Transformers"가 이를 직접적으로 표현한다.

5.2 전체 파이프라인: 2-Stage Training

> 💡 **왜 2-Stage로 나누는가?** 이론적으로는 이미지를 직접 픽셀 단위로 자기회귀 생성할 수 있다 (PixelRNN/CNN이 이 방식). 그러나 256x256 이미지는 256x256x3 = 196,608개의 값으로 이루어져 있다. 이를 하나씩 자기회귀 생성하면 트랜스포머의 연산 비용이 $O(n^2)$이므로 사실상 불가능하다. VQ-GAN의 Stage 1은 이미지를 **16x16 = 256개의 이산 토큰으로 압축**한다. 이러면 트랜스포머가 다뤄야 할 시퀀스 길이가 196,608에서 256으로 **768배 축소**된다. 직관적으로 말하면, Stage 1이 "이미지를 효율적으로 요약"하고, Stage 2가 "요약을 생성"하는 분업이다.

Stage 1: VQ-GAN 훈련 (인코더, 디코더, 코드북, 판별자)

```
이미지 x → [CNN Encoder] → 연속 피처 맵 zₑ → [Quantize] → 이산 코드 zq → [CNN Decoder
                                                  ↕
                                              [Codebook]
                                              (K개 벡터)
```

Stage 2: Autoregressive Prior 훈련 (트랜스포머 디코더)

```
[코드북 인덱스 시퀀스] → [Transformer Decoder] → 다음 코드 예측
```

Stage 1과 Stage 2는 **완전히 독립적으로 훈련**된다. Stage 1이 완료된 후 모델을 동결(freeze)하고, Stage 1으로 추출한 코드북 인덱스들을 Stage 2의 훈련 데이터로 사용한다.

5.3 Stage 1: VQ-GAN 상세

VQ-VAE와 동일한 부분

이미지 x를 CNN 인코더에 통과시키면 **연속값 피처 맵** $z_e(x)$가 나온다. 이를 코드북 $\mathcal{E} = \{e_1, e_2, \ldots, e_K\}$의 벡터 중 가장 가까운 것으로 **양자화(quantize)** (연속값을 유한한 이산값 집합 중 하나로 대응시키는 연산 -- 아날로그 신호를 디지털로 변환하는 것과 동일한 원리)한다:

$$z_q = e_k, \quad \text{where } k = \arg\min_j \|z_e(x) - e_j\|_2$$

이 수식을 풀어서 설명하면:

- 인코더 출력의 한 공간 위치에서 나온 벡터 z_e가 있다 (예: 512차원)
- 코드북에는 K개의 512차원 벡터 $\{e_1, e_2, \ldots, e_K\}$가 저장되어 있다
- 모든 코드북 벡터와의 **유클리드 거리** $\|z_e - e_j\|_2$를 계산한다
- 거리가 **가장 가까운** 코드북 벡터의 인덱스 k를 선택한다 ($\arg\min$)
- z_e를 그 코드북 벡터 e_k로 **대체**한다 → 이것이 양자화

코드북의 각 엔트리는 채널 방향의 벡터이다. 예를 들어 인코더 출력이 $16 \times 16 \times 512$이면, 각 공간 위치의 512차원 벡터를 코드북의 512차원 벡터와 비교하여 가장 가까운 인덱스를 할당한다. 결과적으로 16×16개의 코드북 인덱스를 얻는다.

> 💡 **왜 벡터 양자화(Vector Quantization)를 사용하는가?** 핵심 목표는 이미지를 트랜스포머로 생성하는 것인데, 트랜스포머는 **이산적 토큰**(단어, 서브워드 등)에 대한 확률 분포를 출력하도록 설계되었다. 연속값 피처 맵에 대해서는 소프트맥스를 적용할 수 없다. 벡터 양자화는 연속값 피처를 **유한한 코드북 인덱스**로 변환하여, 이미지를 마치 "시각적 단어들의 문장"처럼 표현할 수 있게 한다. 코드북 크기가 1024이면, 각 위치는 1024개 "시각적 단어" 중 하나로 표현되고, 트랜스포머는 1024-way 분류 문제를 풀면 된다.
>
> 만약 양자화 없이 연속값 피처를 직접 생성하려 하면? 트랜스포머가 각 위치에서 512차원 연속 벡터를 회귀(regression)해야 하는데, 이는 분류보다 훨씬 어렵고 불안정하다.

VQ-VAE 손실 (복습)

$$\mathcal{L}_{\text{VQ}} = \underbrace{\|x - \hat{x}\|^2}_{\text{Reconstruction}} + \underbrace{\|sg[z_e] - e\|^2}_{\text{Codebook}} + \underbrace{\beta\|z_e - sg[e]\|^2}_{\text{Commitment}}$$

각 항을 상세히 분석하면:

- **Reconstruction Loss** $\|x - \hat{x}\|^2$: 원본 이미지 x와 복원 이미지 $\hat{x}$ 사이의 MSE. 이 항은 인코더-디코더 파이프라인 전체가 정보를 잘 보존하도록 한다.
- **Codebook Loss** $\|sg[z_e] - e\|^2$: sg는 **stop gradient** (그래디언트를 차단하는 연산 -- 이 항의 그래디언트가 인코더로 역전파되지 않음)를 의미한다. 이 항은 **코드북 벡터 e만 업데이트**한다 -- 코드북 벡터를 인코더 출력 방향으로 이동시킨다. 직관적으로, "코드북이 인코더를 따라간다".
- **Commitment Loss** $\beta\|z_e - sg[e]\|^2$: 반대로, 이 항은 **인코더 출력 z_e만 업데이트**한다 -- 인코더 출력이 코드북 벡터 근처에 머물도록 한다. 직관적으로, "인코더가 코드북 근처에 머물러야 한다". β는 이 제약의 강도를 조절한다 (논문에서 $\beta = 0.25$).

> 💡 **왜 Codebook Loss와 Commitment Loss가 모두 필요한가?** 양자화 연산 $\arg\min$은 **미분 불가능(non-differentiable)**하다. 따라서 Reconstruction Loss의 그래디언트가 양자화를 통해 인코더로 직접 역전파될 수 없다. 이 문제를 해결하기 위해: (1) 코드북은 Codebook Loss로 별도 업데이트하고, (2) 인코더는 Commitment Loss로 별도 업데이트하며, (3) **Straight-Through Estimator** (forward

> pass에서는 양자화된 z_q를 사용하지만, backward pass에서는 z_q의 그래디언트를 z_e에 그대로 복사하는 트릭)를 사용하여 Reconstruction Loss의 그래디언트도 인코더로 전달한다. 코드에서 이것은 `z_q = z_e + (z_q - z_e).detach()`로 구현된다 -- forward에서는 z_q를 사용하지만, `.detach()`로 인해 backward에서는 z_e의 그래디언트만 계산된다.

VQ-GAN의 추가 요소: Adversarial Loss

> 💡 **왜 VQ-VAE에 GAN 손실을 추가하는가? -- Pixel Loss의 한계** VQ-VAE의 Reconstruction Loss는 **픽셀 단위 MSE**(L2 distance)이다. 이 손실의 문제점은 "블러링(blurring)"이다. 여러 가능한 복원 결과의 **평균**이 MSE를 최소화하는데, 이미지의 평균은 블러한 이미지이다. 예를 들어, 머리카락의 정확한 위치는 불확실하므로, MSE만 사용하면 모든 가능한 머리카락 위치의 평균인 뿌연 이미지가 출력된다. **Perceptual Loss** (VGG 같은 사전 학습된 네트워크의 중간 피처 공간에서의 거리 -- 픽셀 차이가 아닌 "지각적 유사성"을 측정)나 **Adversarial Loss**는 이 블러링 문제를 해결한다. Adversarial Loss는 "복원된 이미지가 실제 이미지 분포에서 온 것처럼 보여야 한다"고 강제하므로, 블러한 이미지(실제 이미지 분포에 없음)는 벌칙을 받는다.

VQ-VAE에 **PatchGAN Discriminator**를 달아 적대적 손실을 추가:

$$\mathcal{L}_{\text{GAN}} = \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(\hat{x}))]$$

- 복원된 이미지 $\hat{x}$가 진짜처럼 보이도록 추가 훈련
- PatchGAN: 이미지를 패치 단위로 분할하여 각 영역별로 진짜/가짜 판정 (CycleGAN에서 사용한 것과 동일한 구조)

Pixel Loss vs Perceptual Loss vs Adversarial Loss 비교

손실 함수	측정 대상	장점	단점
Pixel Loss (L1/L2)	픽셀 값 차이	구현 간단, 안정적	블러링, 인간 지각과 불일치
Perceptual Loss	VGG 피처 공간 거리	지각적 유사성 반영	사전학습 모델 필요, 스타일 편향 가능
Adversarial Loss	분포 수준 유사성	선명한 결과, 사실적 질감	훈련 불안정, mode collapse 위험

VQ-GAN은 Pixel Loss(Reconstruction)와 Adversarial Loss를 **결합**하여, 안정성과 선명도를 모두 확보한다. 실제 논문에서는 Perceptual Loss도 함께 사용한다.

전체 Stage 1 손실

$$\boxed{\mathcal{L}_{\text{Stage 1}} = \mathcal{L}_{\text{VQ}} + \lambda \cdot \mathcal{L}_{\text{GAN}}}$$

- λ: 하이퍼파라미터 (GAN 손실의 비중)
- $\lambda = 0$이면 → 그냥 VQ-VAE

> 저자의 견해: GAN 손실은 이미지 품질을 약간 개선하지만, 핵심은 아니다. VQ-VAE만으로도 기본은 동작한다.

> ⚠️ **실전 팁**: VQ-GAN 논문에서는 적응적 가중치(adaptive weight) λ를 사용한다. Generator의 마지막 레이어 그래디언트 크기를 기준으로 λ를 자동 조절하여 VQ 손실과 GAN 손실의 균형을 맞춘다. 구체적으로, $\lambda = \frac{\|\nabla_{G_L}\mathcal{L}_{\text{VQ}}\|}{\|\nabla_{G_L}\mathcal{L}_{\text{GAN}}\|+\delta}$ 형태로 계산한다 (G_L은 디코더의 마지막 레이어, δ는 안정화 상수). 이렇게 하면 VQ 손실과 GAN 손실의 그래디언트 크기가 비슷하게 유지되어 훈련이 안정적이다.

Stage 1 완료 후 필수 작업: 코드북 인덱스 추출

Stage 1 훈련이 끝나면, **모든 훈련 이미지에 대해 코드북 인덱스 맵을 추출**해야 한다:

$$\text{모든 이미지 } x_i \xrightarrow{\text{Encoder + Quantize}} \text{코드북 인덱스 맵 } s_i$$

이 과정은 학습이 아니라 **단순 추론(inference)**이다. 각 이미지의 저차원 표현(코드북 인덱스)을 "레이블처럼" 기록해 두는 것이다.

> **핵심 포인트**: 이 추출된 코드북 인덱스 시퀀스가 Stage 2의 훈련 데이터가 된다.

5.4 코드북 인덱스 시퀀스의 이해

> 💡 **이 섹션이 중요한 이유** Stage 1에서 Stage 2로 넘어가는 핵심 전환점이다. Stage 1이 만드는 "코드북 인덱스 시퀀스"가 정확히 무엇인지 이해해야 Stage 2의 자기회귀 생성을 이해할 수 있다. 여기서 확립되는 **이미지 = 토큰 시퀀스**라는 등식이 DALL-E, Stable Diffusion 등 이후 모든 모델의 기초가 된다.

용어 정리

동일한 것을 가리키는 여러 이름:

이름	설명
코드북 인덱스 시퀀스	코드북의 인덱스 번호들의 나열
이미지 토큰 시퀀스	이미지를 토큰화한 시퀀스
코드북 인덱스 맵	2D 형태로 배열 (16×16 등)
이미지 토큰 맵	동일

자연어 토큰과의 대응

자연어	이미지
Vocabulary (예: 50,000개 단어)	Codebook (예: 1,024개 코드)
단어 → Vocabulary 인덱스	피처 벡터 → Codebook 인덱스
문장 = 단어 인덱스 시퀀스	이미지 = 코드 인덱스 시퀀스

코드북 크기가 1,024이고 이미지가 16×16으로 압축되면, 하나의 이미지는 **256개의 토큰**으로 표현된다. 각 토큰은 1~1,024 중 하나의 값.

> 💡 **압축률을 계산해보자** 원본 이미지: 256x256 픽셀, RGB 3채널, 각 8비트 = 256 x 256 x 3 x 8 = **1,572,864 비트** VQ-GAN 압축: 16x16 인덱스, 각 인덱스는 $\log_2(1024) = 10$비트 = 16 x 16 x 10 = **2,560 비트** 압축률: 1,572,864 / 2,560 = **약 614배 압축**. 물론 이는 lossy compression이므로 완벽한 복원은 불가능하지만, VQ-GAN의 디코더가 2,560 비트의 정보만으로 사실적인 이미지를 복원할 수 있다는 점이 놀랍다. 이것이 가능한 이유는 코드북 벡터 자체가 풍부한 시각 정보를 담고 있기 때문이다 -- 인덱스는 "어떤 코드북 벡터를 사용하라"는 포인터일 뿐이고, 실제 정보는 512차원 코드북 벡터에 저장되어 있다.

5.5 Stage 2: Autoregressive Prior (트랜스포머 디코더)

핵심 전환

> **이제부터 이미지는 다루지 않는다. 코드북 인덱스 시퀀스만 다룬다.**

이미지를 이산적 토큰 시퀀스로 변환했으므로, **자연어 생성과 동일한 방식**으로 이미지를 생성할 수 있다. 이것이 VQ-GAN의 가장 혁신적인 통찰이다: **이미지 생성을 언어 모델링으로 환원(reduce)**한 것이다. NLP에서 수년간 발전시켜 온 트랜스포머 기술, 훈련 기법, 스케일링 법칙을 그대로 활용할 수 있다.

훈련 방법

그라운드 트루스 코드 시퀀스: $[s_1, s_2, s_3, \ldots, s_{256}]$ (각 $s_i \in \{1, 2, \ldots, 1024\}$)

이전 토큰들을 조건으로 **다음 토큰을 예측** -- 이것을 **자기회귀적(autoregressive) 생성**이라 한다 (자기 자신의 이전 출력을 다음 입력으로 사용하여 순차적으로 생성):

$$p(s_i \mid s_1, s_2, \ldots, s_{i-1})$$

예를 들어 시퀀스가 $[1, 42, 3, 94, 60, 22, 57, \ldots]$일 때:

```
입력: [1, 42, 3, 94, 60, 22]  →  예측: softmax 분포 (1024차원)  →  정답: 57
```

- 출력: 1024개 코드에 대한 **소프트맥스 확률 분포** -- 각 코드가 다음에 올 확률
- 정답: 원핫 벡터 (57번만 1, 나머지 0)
- 손실: **Cross-Entropy Loss** (예측 확률 분포와 정답 분포 사이의 거리)

$$\mathcal{L}_{\text{prior}} = -\sum_{i=1}^{N} \log p(s_i \mid s_{<i})$$

이 수식을 풀어서 설명하면: 각 위치 i에서 트랜스포머가 출력한 소프트맥스 분포에서, 정답 토큰 s_i에 배정된 확률의 로그값을 취한다. 이 값이 클수록(즉 정답에 높은 확률을 배정할수록) 손실이 작아진다. 모든 위치에 대해 합산한 것이 전체 손실이다.

이것은 GPT가 다음 단어를 예측하는 것과 **완전히 동일한 구조**이다. GPT의 vocabulary가 50,000개 단어인 것처럼, 여기서는 vocabulary가 1,024개 시각 코드이다.

Teacher Forcing으로 훈련

- **Teacher Forcing** (훈련 시 모델의 예측값이 아닌 정답값을 다음 스텝의 입력으로 사용하는 기법): 예측이 틀려도 **정답 토큰을 다음 입력으로 사용** → 훈련이 안정적이고 빠르다. 만약 이전 예측이 틀렸을 때 그 틀린 값을 다음 입력으로 쓰면, 오류가 누적되어 훈련이 발산할 수 있다.
- **Causal Mask** (삼각 행렬 형태의 마스크로, 각 토큰이 자기 자신과 이전 토큰에만 어텐션할 수 있게 제한) 적용: s_i 예측 시 $s_{i+1}, s_{i+2}, \ldots$ 를 보지 못하도록 마스킹 → 추론 시에는 미래 토큰이 존재하지 않으므로, 훈련과 추론의 조건을 일치시키기 위해 필요
- 자연어 트랜스포머 디코더 훈련과 동일

직접 의존성 (실제 참조 영역)

실제로 각 토큰을 생성할 때 앞의 모든 토큰을 직접 사용하는 것은 아니다. PixelCNN처럼 **이웃 영역**의 토큰을 조건으로 사용하고, 더 먼 토큰은 간접적으로 영향을 받는다:

```
현재 생성할 토큰:  [?]
직접 참조 영역:    ↑ (위), ← (왼쪽), ↖ (대각선)
간접 참조:        이전에 생성된 토큰들의 영향이 전파
```

하지만 트랜스포머의 Self-Attention 덕분에, 실제로는 **앞의 모든 토큰에 어텐션**을 수행하므로 장거리 의존성도 잘 포착한다.

첫 번째 토큰의 문제

첫 번째 토큰 s_1은 **조건 없이** 생성해야 한다. 이는 다소 랜덤한 결과를 낳을 수 있다. 이를 해결하기 위해 **조건(condition)**을 부여하는 방법이 도입된다.

5.6 조건부 생성 (Conditional Generation)

> 💡 **왜 조건부 생성이 필요한가?** 무조건 생성(unconditional generation)은 "랜덤한 이미지"를 만든다. 실용적으로는 **원하는 내용의 이미지**를 만들어야 한다: "바나나 사진을 만들어라", "이 엣지맵에 맞는 사진을 만들어라" 등. 조건부 생성은 트랜스포머에게 "힌트"를 제공하여, 그 힌트에 부합하는 이미지를 생성하게 한다. VQ-GAN의 장점은, 조건이 이미지든 텍스트든 **모두 토큰 시퀀스로 통일**되므로, 동일한 프레임워크로 다양한 조건부 생성을 구현할 수 있다는 것이다.

방법: 조건 토큰을 프리픽스로 연결

조건 이미지(예: 엣지 맵)의 코드북 인덱스 시퀀스를 **정답 이미지 토큰 앞에 이어 붙여** 단일 시퀀스로 만든다:

```
[조건 토큰₁, 조건 토큰₂, ..., 조건 토큰₁₆ | 이미지 토큰₁, 이미지 토큰₂, ..., 이미지 토큰
```

조건 이미지 준비:

1. 원본 이미지에서 **전처리**(엣지 추출 등)로 조건 이미지 생성 (고전적 이미지 처리)
2. 조건 이미지를 VQ-GAN에 통과시켜 코드북 인덱스 시퀀스 추출
3. 원본 이미지의 코드북 인덱스 시퀀스 앞에 연결

이렇게 하면 트랜스포머가 **조건 토큰을 어텐션**하면서 이미지 토큰을 생성하므로, 첫 번째 이미지 토큰부터 의미 있는 생성이 가능하다.

텍스트 조건도 동일한 방식

이미지 조건 대신 **텍스트 토큰**을 앞에 붙여도 된다:

```
["yellow", "bunch", "of", "bananas" | 이미지 토큰₁, 이미지 토큰₂, ...]
```

- 텍스트 토큰은 vocabulary 인덱스로 이미 이산화되어 있으므로 별도 처리 불필요
- VQ-GAN 논문에서는 ImageNet 레이블(클래스명)을 조건으로 사용

> **핵심 통찰**: 조건이 이미지든, 텍스트든, 레이블이든 — 모두 **토큰 시퀀스로 변환되어 연결**되면, 트랜스포머에게는 동일한 문제가 된다.

텍스트 조건 이미지 생성 예시

"Yellow bunch of bananas"라는 레이블이 주어지면:

훈련 시 (Teacher Forcing):

```
입력: [yellow, bunch, of, bananas, s₁(정답), s₂(정답), ...]
목표:                              [s₁,         s₂,         s₃, ...]
```

각 위치에서 소프트맥스 분포를 출력하고, 정답과 Cross-Entropy Loss를 계산하여 트랜스포머를 훈련.

추론 시 (Autoregressive Generation):

```
입력: [yellow, bunch, of, bananas]
→ s₁* 생성 (소프트맥스에서 샘플링)
→ [yellow, bunch, of, bananas, s₁*]
→ s₂* 생성
→ [yellow, bunch, of, bananas, s₁*, s₂*]
→ ... 반복 ...
→ 최종: [s₁*, s₂*, ..., s₂₅₆*]
→ 코드북에서 벡터 복원 → VQ-GAN 디코더 → 이미지
```

5.7 조건 부여 방식의 발전

방식 1: 단순 연결 (Concatenation)

$$\text{sequence} = [\text{condition tokens}; \text{image tokens}]$$

조건 토큰과 이미지 토큰을 **동등한 레벨**로 하나의 시퀀스에 연결. 트랜스포머 디코더가 전체를 Self-Attention.

- 사용 모델: VQ-GAN, DALL-E (초기)
- 장점: 구현이 간단하고, 별도의 인코더 네트워크가 필요 없다
- 단점: 조건 토큰과 이미지 토큰이 **동일한 임베딩 공간**에서 처리되므로, 조건의 의미를 충분히 파악하기 어렵다. 또한 조건 토큰이 길어지면 전체 시퀀스 길이가 증가하여 Self-Attention의 $O(n^2)$ 비용이 급증한다.

방식 2: 인코더 + Cross-Attention

조건을 **별도의 인코더**로 먼저 처리한 뒤, 트랜스포머 디코더와 **Cross-Attention** (디코더의 각 위치가 인코더 출력의 모든 위치에 어텐션하는 메커니즘 -- Self-Attention과 달리, Query는 디코더에서, Key와 Value는 인코더에서 온다):

```
조건 텍스트 → [Encoder] → 조건 임베딩
                                  ↓ (Cross-Attention)
이미지 토큰 → [Transformer Decoder] → 다음 이미지 토큰 예측
```

- 인코더가 문장의 **전체 의미를 사전에 파악**해서 넣어주므로 성능이 더 좋음
- 날것의 토큰이 아닌, **완벽하게 처리된 표현**이 트랜스포머 디코더로 들어감
- 사용 모델: CogView, DALL-E 2, Imagen 등 최신 모델

> 💡 **방식 1 vs 방식 2의 핵심 차이** 방식 1에서 조건 토큰 "yellow bunch of bananas"는 각 단어가 개별적으로 이미지 토큰과 어텐션된다. 트랜스포머는 "yellow"와 "bananas"가 하나의 구를 이룬다는 것을 Self-Attention을 통해 **스스로 파악해야** 한다. 방식 2에서는 인코더가 먼저 "yellow bunch of bananas"라는 구의 **전체 의미를 이해**한 후, 그 이해를 압축된 표현으로 디코더에 전달한다. 이미지 생성 디코더는 "무엇을 그려야 하는가?"를 이미 파악된 상태에서 받으므로, "어떻게 그릴 것인가?"에만 집중할 수 있다. 이것이 분업(division of labor)의 효과이다.

> DALL-E (2021)에서는 방식 1을 사용했지만, 이는 CLIP이 아직 없었기 때문. DALL-E 2부터는 CLIP 인코더로 텍스트를 먼저 처리한 뒤 Cross-Attention하는 방식 2를 채택.

5.8 훈련/추론 과정 전체 정리

전체 훈련 과정

```
[Stage 1: VQ-GAN 훈련]
모든 이미지 x → CNN Encoder → Quantize → 코드북 인덱스 → CNN Decoder → x̂
                                                                    ↓
                                                          PatchGAN Discriminator
Loss = L_VQ + λ·L_GAN → 인코더, 디코더, 코드북, 판별자 업데이트

[Stage 1 완료 후: 코드 추출]
모든 훈련 이미지 → Encoder + Quantize → 코드북 인덱스 시퀀스 기록

[Stage 2: Transformer Decoder 훈련]
코드북 인덱스 시퀀스 → Transformer Decoder → 다음 코드 예측
Loss = Cross-Entropy → 트랜스포머 업데이트
```

추론 (이미지 생성) 과정

```
(선택) 조건 입력 → 토큰화
→ Transformer Decoder: 코드 인덱스를 하나씩 자기회귀적 생성
→ 생성된 코드 인덱스 시퀀스
→ 코드북에서 벡터 복원
→ VQ-GAN Decoder → 최종 이미지
```

5.9 PyTorch 구현

VQ-GAN 전체 파이프라인 (Stage 1 + Stage 2)

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


# ============================================================
# Stage 1: VQ-GAN (인코더 + 양자화 + 디코더 + PatchGAN)
# ============================================================

class VectorQuantizer(nn.Module):
    """
    벡터 양자화 모듈.
    인코더 출력의 각 공간 위치 벡터를 코드북에서 가장 가까운 벡터로 대체한다.
    Straight-Through Estimator를 사용하여 역전파를 가능하게 한다.

    Args:
        num_embeddings: 코드북 크기 K (예: 1024)
        embedding_dim: 코드 벡터 차원 (예: 256)
        beta: Commitment loss 가중치 (기본 0.25)
    """
    def __init__(self, num_embeddings=1024, embedding_dim=256, beta=0.25):
        super(VectorQuantizer, self).__init__()
        self.K = num_embeddings
        self.D = embedding_dim
        self.beta = beta

        # 코드북: K개의 D차원 벡터
        self.embedding = nn.Embedding(num_embeddings, embedding_dim)
        self.embedding.weight.data.uniform_(-1.0 / num_embeddings,
                                            1.0 / num_embeddings)

    def forward(self, z_e):
        """
        Args:
            z_e: 인코더 출력 (B, D, H, W) — 연속값 피처 맵
        Returns:
            z_q: 양자화된 피처 맵 (B, D, H, W)
            loss: VQ 손실 (codebook + commitment)
            indices: 코드북 인덱스 (B, H, W)
        """
        B, D, H, W = z_e.shape

        # (B, D, H, W) → (B*H*W, D) 로 변환하여 각 위치별 벡터 추출
        z_e_flat = z_e.permute(0, 2, 3, 1).contiguous().view(-1, D)  # (B*H*W, D)

        # 각 벡터와 코드북의 모든 벡터 사이의 L2 거리 계산
        # ||z_e - e_k||^2 = ||z_e||^2 + ||e_k||^2 - 2 * z_e · e_k
        distances = (
            torch.sum(z_e_flat ** 2, dim=1, keepdim=True)            # (B*H*W, 1)
            + torch.sum(self.embedding.weight ** 2, dim=1)            # (K,)
            - 2 * torch.matmul(z_e_flat, self.embedding.weight.t())   # (B*H*W, K)
        )

        # 가장 가까운 코드북 인덱스 선택
        indices = torch.argmin(distances, dim=1)  # (B*H*W,)

        # 코드북에서 벡터 조회
        z_q_flat = self.embedding(indices)  # (B*H*W, D)
        z_q = z_q_flat.view(B, H, W, D).permute(0, 3, 1, 2)  # (B, D, H, W)

        # VQ 손실 계산
```

```python
        codebook_loss = F.mse_loss(z_q.detach(), z_e)    # Commitment: 인코더→코드북
        commitment_loss = F.mse_loss(z_q, z_e.detach())  # Codebook: 코드북→인코더

        loss = codebook_loss + self.beta * commitment_loss

        # Straight-Through Estimator: forward는 z_q, backward는 z_e의 그래디언트 사용
        z_q = z_e + (z_q - z_e).detach()

        indices = indices.view(B, H, W)
        return z_q, loss, indices


class VQGANEncoder(nn.Module):
    """
    VQ-GAN 인코더: 이미지를 연속값 피처 맵으로 압축한다.
    예: (B, 3, 256, 256) → (B, 256, 16, 16)

    Args:
        in_channels: 입력 채널 (RGB=3)
        hidden_dims: 중간 채널 수 리스트
        embedding_dim: 출력 피처 차원
    """
    def __init__(self, in_channels=3, hidden_dims=None, embedding_dim=256):
        super(VQGANEncoder, self).__init__()

        if hidden_dims is None:
            hidden_dims = [64, 128, 256]

        layers = []
        prev_channels = in_channels
        for h_dim in hidden_dims:
            layers.extend([
                nn.Conv2d(prev_channels, h_dim, 4, stride=2, padding=1),  # 해상도
                nn.BatchNorm2d(h_dim),
                nn.LeakyReLU(0.2, inplace=True),
            ])
            prev_channels = h_dim

        # 최종 projection → embedding_dim
        layers.append(nn.Conv2d(prev_channels, embedding_dim, 3, padding=1))

        self.encoder = nn.Sequential(*layers)

    def forward(self, x):
        """
        Args:
            x: 입력 이미지 (B, 3, H, W)
        Returns:
            z_e: 연속값 피처 맵 (B, embedding_dim, H/8, W/8)
        """
        return self.encoder(x)


class VQGANDecoder(nn.Module):
    """
    VQ-GAN 디코더: 양자화된 피처 맵에서 이미지를 복원한다.
    예: (B, 256, 16, 16) → (B, 3, 256, 256)

    인코더의 역순 구조 (Transposed Convolution으로 업샘플링)

    Args:
```

```python
    Args:
        embedding_dim: 입력 피처 차원
        hidden_dims: 중간 채널 수 리스트 (인코더의 역순)
        out_channels: 출력 채널 (RGB=3)
    """
    def __init__(self, embedding_dim=256, hidden_dims=None, out_channels=3):
        super(VQGANDecoder, self).__init__()

        if hidden_dims is None:
            hidden_dims = [256, 128, 64]

        layers = [nn.Conv2d(embedding_dim, hidden_dims[0], 3, padding=1)]

        for i in range(len(hidden_dims) - 1):
            layers.extend([
                nn.ConvTranspose2d(hidden_dims[i], hidden_dims[i + 1],
                                   4, stride=2, padding=1),  # 해상도 2배
                nn.BatchNorm2d(hidden_dims[i + 1]),
                nn.ReLU(inplace=True),
            ])

        # 최종 출력 레이어
        layers.extend([
            nn.ConvTranspose2d(hidden_dims[-1], out_channels,
                               4, stride=2, padding=1),
            nn.Tanh(),  # [-1, 1] 범위
        ])

        self.decoder = nn.Sequential(*layers)

    def forward(self, z_q):
        """
        Args:
            z_q: 양자화된 피처 맵 (B, embedding_dim, H', W')
        Returns:
            x_recon: 복원 이미지 (B, 3, H, W)
        """
        return self.decoder(z_q)


class VQGAN(nn.Module):
    """
    VQ-GAN Stage 1 전체 모델.
    인코더 → 벡터 양자화 → 디코더 파이프라인.

    Args:
        in_channels: 입력 채널 수
        embedding_dim: 코드 벡터 차원
        num_embeddings: 코드북 크기 K
        beta: Commitment loss 가중치
    """
    def __init__(self, in_channels=3, embedding_dim=256,
                 num_embeddings=1024, beta=0.25):
        super(VQGAN, self).__init__()

        self.encoder = VQGANEncoder(in_channels, embedding_dim=embedding_dim)
        self.quantizer = VectorQuantizer(num_embeddings, embedding_dim, beta)
        self.decoder = VQGANDecoder(embedding_dim, out_channels=in_channels)

    def forward(self, x):
        """
        .
```

```
        Args:
            x: 입력 이미지 (B, 3, H, W)
        Returns:
            x_recon: 복원 이미지 (B, 3, H, W)
            vq_loss: VQ 손실
            indices: 코드북 인덱스 (B, H', W')
        """
        z_e = self.encoder(x)                      # 인코딩
        z_q, vq_loss, indices = self.quantizer(z_e)  # 양자화
        x_recon = self.decoder(z_q)                # 디코딩
        return x_recon, vq_loss, indices

    @torch.no_grad()
    def encode_to_indices(self, x):
        """
        추론 전용: 이미지를 코드북 인덱스로 변환.
        Stage 1 훈련 완료 후, Stage 2 훈련 데이터를 생성할 때 사용.

        Args:
            x: 입력 이미지 (B, 3, H, W)
        Returns:
            indices: 코드북 인덱스 (B, H', W')
        """
        z_e = self.encoder(x)
        _, _, indices = self.quantizer(z_e)
        return indices

    @torch.no_grad()
    def decode_from_indices(self, indices):
        """
        추론 전용: 코드북 인덱스에서 이미지 복원.
        Stage 2에서 생성된 인덱스 시퀀스를 이미지로 변환할 때 사용.

        Args:
            indices: 코드북 인덱스 (B, H', W')
        Returns:
            x_recon: 복원 이미지 (B, 3, H, W)
        """
        B, H, W = indices.shape
        # 인덱스에서 코드북 벡터 조회
        z_q = self.quantizer.embedding(indices)              # (B, H, W, D)
        z_q = z_q.permute(0, 3, 1, 2).contiguous()          # (B, D, H, W)
        x_recon = self.decoder(z_q)
        return x_recon
```

Stage 1 훈련 루프 (VQ + GAN 결합)

```python
def train_vqgan_stage1(vqgan, discriminator, dataloader, num_epochs, device,
                       lr=1e-4, lambda_gan=0.1):
    """
    VQ-GAN Stage 1 훈련: VQ-VAE 손실 + GAN 손실로 인코더/디코더/코드북/판별자를 훈련한

    Args:
        vqgan: VQGAN 모델 (인코더 + 양자화 + 디코더)
        discriminator: PatchGAN Discriminator
        dataloader: 이미지 DataLoader
        num_epochs: 에폭 수
        device: 'cuda' 또는 'cpu'
        lr: 학습률
        lambda_gan: GAN 손실 가중치
    """
    opt_vqgan = torch.optim.Adam(vqgan.parameters(), lr=lr, betas=(0.5, 0.999))
    opt_disc = torch.optim.Adam(discriminator.parameters(), lr=lr, betas=(0.5, 0.9

    for epoch in range(num_epochs):
        for batch_idx, (imgs, _) in enumerate(dataloader):
            imgs = imgs.to(device)

            # ---- VQ-GAN (Generator) 훈련 ----
            opt_vqgan.zero_grad()

            x_recon, vq_loss, _ = vqgan(imgs)

            # Reconstruction Loss (L1 또는 L2)
            recon_loss = F.mse_loss(x_recon, imgs)

            # GAN Loss: 복원 이미지가 진짜로 판별되도록
            disc_fake = discriminator(x_recon)
            gan_loss = -torch.mean(disc_fake)  # Hinge Loss 변형

            # 전체 Generator Loss
            g_loss = recon_loss + vq_loss + lambda_gan * gan_loss
            g_loss.backward()
            opt_vqgan.step()

            # ---- Discriminator 훈련 ----
            opt_disc.zero_grad()

            disc_real = discriminator(imgs)
            disc_fake = discriminator(x_recon.detach())

            # Hinge Loss
            d_loss = torch.mean(F.relu(1.0 - disc_real)) + torch.mean(F.relu(1.0 +
            d_loss.backward()
            opt_disc.step()

            if batch_idx % 100 == 0:
                print(f"[Epoch {epoch+1}/{num_epochs}] [Batch {batch_idx}] "
                      f"Recon: {recon_loss.item():.4f} | VQ: {vq_loss.item():.4f}
                      f"G_GAN: {gan_loss.item():.4f} | D: {d_loss.item():.4f}")
```

Stage 2: **자기회귀 트랜스포머 Prior**

```python
class ImageTokenTransformer(nn.Module):
    """
    VQ-GAN Stage 2: 이미지 토큰 시퀀스에 대한 자기회귀적 트랜스포머.
    GPT와 동일한 구조로, 이전 토큰들을 보고 다음 토큰을 예측한다.

    Args:
        vocab_size: 코드북 크기 K (예: 1024)
        seq_len: 이미지 토큰 시퀀스 길이 (예: 16×16 = 256)
        d_model: 트랜스포머 임베딩 차원
        nhead: 멀티헤드 어텐션의 헤드 수
        num_layers: 트랜스포머 디코더 레이어 수
        cond_vocab_size: 조건 토큰의 vocabulary 크기 (조건부 생성 시, None이면 무조건
    """
    def __init__(self, vocab_size=1024, seq_len=256, d_model=512,
                 nhead=8, num_layers=6, cond_vocab_size=None):
        super(ImageTokenTransformer, self).__init__()

        self.vocab_size = vocab_size
        self.seq_len = seq_len

        # 이미지 토큰 임베딩
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        # 위치 임베딩 (학습 가능)
        self.position_embedding = nn.Embedding(seq_len, d_model)

        # 조건 토큰 임베딩 (선택적)
        self.cond_embedding = None
        if cond_vocab_size is not None:
            self.cond_embedding = nn.Embedding(cond_vocab_size, d_model)

        # 트랜스포머 디코더 레이어
        decoder_layer = nn.TransformerDecoderLayer(
            d_model=d_model, nhead=nhead,
            dim_feedforward=d_model * 4,
            dropout=0.1, activation='gelu',
            batch_first=True
        )
        self.transformer = nn.TransformerDecoder(decoder_layer, num_layers=num_lay

        # 출력 projection → 코드북 크기의 logits
        self.output_head = nn.Linear(d_model, vocab_size)

        # Causal Mask 생성: 미래 토큰을 보지 못하도록
        self.register_buffer(
            'causal_mask',
            torch.triu(torch.ones(seq_len, seq_len), diagonal=1).bool()
        )

    def forward(self, indices, cond_tokens=None):
        """
        Args:
            indices: 코드북 인덱스 시퀀스 (B, seq_len)
            cond_tokens: 조건 토큰 (B, cond_len), 선택적
        Returns:
            logits: 각 위치의 다음 토큰 예측 (B, seq_len, vocab_size)
        """
        B, T = indices.shape

        # 토큰 임베딩 + 위치 임베딩
        positions = torch.arange(T, device=indices.device).unsqueeze(0)  # (1, T)
        x = self.token_embedding(indices) + self.position_embedding(positions)  #
```

```python
        # 조건 토큰이 있으면 memory로 사용 (Cross-Attention)
        memory = None
        if cond_tokens is not None and self.cond_embedding is not None:
            memory = self.cond_embedding(cond_tokens)  # (B, cond_len, d_model)

        # Causal Mask 적용
        mask = self.causal_mask[:T, :T]

        if memory is not None:
            x = self.transformer(x, memory, tgt_mask=mask)
        else:
            # memory 없이 Self-Attention만 사용
            x = self.transformer(x, x, tgt_mask=mask)

        # 출력: 각 위치에서 vocab_size 크기의 logits
        logits = self.output_head(x)  # (B, T, vocab_size)
        return logits

    @torch.no_grad()
    def generate(self, num_samples=1, temperature=1.0, top_k=100,
                 cond_tokens=None, device='cuda'):
        """
        자기회귀적 이미지 토큰 생성.

        Args:
            num_samples: 생성할 이미지 수
            temperature: 샘플링 온도 (높을수록 다양, 낮을수록 보수적)
            top_k: Top-K 샘플링 (상위 K개만 고려)
            cond_tokens: 조건 토큰 (num_samples, cond_len)
            device: 디바이스
        Returns:
            generated: 생성된 코드북 인덱스 시퀀스 (num_samples, seq_len)
        """
        generated = torch.zeros(num_samples, 0, dtype=torch.long, device=device)

        for i in range(self.seq_len):
            if generated.shape[1] == 0:
                # 첫 토큰: 랜덤 시작 또는 특수 토큰
                start_token = torch.zeros(num_samples, 1, dtype=torch.long, device
                logits = self.forward(start_token, cond_tokens)
            else:
                logits = self.forward(generated, cond_tokens)

            # 마지막 위치의 logits만 사용
            next_logits = logits[:, -1, :] / temperature  # (B, vocab_size)

            # Top-K 필터링
            if top_k > 0:
                top_k_values, _ = torch.topk(next_logits, top_k)
                threshold = top_k_values[:, -1].unsqueeze(1)
                next_logits[next_logits < threshold] = float('-inf')

            # 소프트맥스 → 샘플링
            probs = F.softmax(next_logits, dim=-1)
            next_token = torch.multinomial(probs, 1)  # (B, 1)

            generated = torch.cat([generated, next_token], dim=1)

        return generated
```

```python
def train_vqgan_stage2(transformer, vqgan, dataloader, num_epochs, device, lr=3e-4
    """
    VQ-GAN Stage 2 훈련: 트랜스포머 Prior를 Cross-Entropy Loss로 훈련한다.

    이미 훈련된 VQ-GAN (Stage 1)으로 모든 이미지를 코드북 인덱스로 변환한 뒤,
    이 인덱스 시퀀스에 대해 자기회귀적 예측을 학습한다.

    Args:
        transformer: ImageTokenTransformer 모델
        vqgan: 훈련 완료된 VQGAN 모델 (freeze됨)
        dataloader: 이미지 DataLoader
        num_epochs: 에폭 수
        device: 디바이스
        lr: 학습률
    """
    optimizer = torch.optim.Adam(transformer.parameters(), lr=lr)
    vqgan.eval()  # VQ-GAN은 고정

    for epoch in range(num_epochs):
        for batch_idx, (imgs, _) in enumerate(dataloader):
            imgs = imgs.to(device)

            # Step 1: VQ-GAN으로 코드북 인덱스 추출 (추론만, 학습 안 함)
            with torch.no_grad():
                indices = vqgan.encode_to_indices(imgs)  # (B, H', W')
                B, H, W = indices.shape
                indices_flat = indices.view(B, -1)       # (B, H'*W') = (B, seq_le

            # Step 2: 트랜스포머로 다음 토큰 예측
            # 입력: [s_1, s_2, ..., s_{N-1}], 타겟: [s_2, s_3, ..., s_N]
            input_tokens = indices_flat[:, :-1]   # (B, seq_len - 1)
            target_tokens = indices_flat[:, 1:]   # (B, seq_len - 1)

            optimizer.zero_grad()

            logits = transformer(input_tokens)    # (B, seq_len - 1, vocab_size)

            # Cross-Entropy Loss
            loss = F.cross_entropy(
                logits.reshape(-1, transformer.vocab_size),  # (B*(seq_len-1), K)
                target_tokens.reshape(-1)                     # (B*(seq_len-1),)
            )

            loss.backward()
            optimizer.step()

            if batch_idx % 100 == 0:
                print(f"[Epoch {epoch+1}/{num_epochs}] [Batch {batch_idx}] "
                      f"CE Loss: {loss.item():.4f}")
```

전체 추론 파이프라인

```python
@torch.no_grad()
def generate_images(vqgan, transformer, num_images=4, temperature=1.0,
                    top_k=100, device='cuda'):
    """
    VQ-GAN + Transformer로 이미지를 생성하는 전체 파이프라인.

    Step 1: Transformer가 코드북 인덱스 시퀀스를 자기회귀적으로 생성
    Step 2: VQ-GAN 디코더가 인덱스 시퀀스를 이미지로 변환

    Args:
        vqgan: 훈련 완료된 VQGAN 모델
        transformer: 훈련 완료된 ImageTokenTransformer 모델
        num_images: 생성할 이미지 수
        temperature: 샘플링 온도
        top_k: Top-K 샘플링
        device: 디바이스
    Returns:
        images: 생성된 이미지 (num_images, 3, H, W)
    """
    vqgan.eval()
    transformer.eval()

    # Step 1: 트랜스포머로 코드북 인덱스 생성
    generated_indices = transformer.generate(
        num_samples=num_images,
        temperature=temperature,
        top_k=top_k,
        device=device
    )  # (num_images, seq_len)

    # seq_len을 2D 인덱스 맵으로 reshape (예: 256 → 16×16)
    H = W = int(generated_indices.shape[1] ** 0.5)
    indices_2d = generated_indices.view(num_images, H, W)

    # Step 2: VQ-GAN 디코더로 이미지 복원
    images = vqgan.decode_from_indices(indices_2d)

    return images
```

---

6. VQ-GAN에서 DALL-E로의 연결

> 💡 **왜 이 연결이 중요한가?** VQ-GAN은 "이미지를 이산 토큰으로 표현하면 트랜스포머로 생성할 수 있다"는 원리를 증명했다. DALL-E는 이 원리를 **텍스트 조건부 이미지 생성**이라는 실용적 응용으로 확장한 것이다. VQ-GAN → DALL-E → DALL-E 2 → Stable Diffusion으로 이어지는 계보를 이해하면, 현재 이미지 생성 기술의 발전 과정을 한눈에 파악할 수 있다. 특히 VQ-GAN의 저자(Esser, Rombach, Ommer)가 이후 **Stable Diffusion(Latent Diffusion Model)**의 저자이기도 하다는 점에서, VQ-GAN의 "이미지를 저차원 잠재 공간으로 압축한 뒤 생성한다"는 아이디어가 Diffusion Model에도 직접적으로 계승되었음을 알 수 있다.

6.1 DALL-E (2021)

> 📄 **논문**: Ramesh et al., "Zero-Shot Text-to-Image Generation", OpenAI, 2021

DALL-E는 VQ-GAN과 **거의 동일한 구조**:

구성 요소	VQ-GAN	DALL-E
Stage 1	VQ-GAN 훈련	"Visual Codebook 훈련"
Stage 2	Transformer Decoder 훈련	"Prior 훈련"
조건	이미지/레이블	**텍스트** (자연어 문장)
조건 부여 방식	단순 연결 (Concat)	단순 연결 (Concat)

핵심 차이: DALL-E는 **대규모 텍스트-이미지 데이터셋** (2.5억 쌍)으로 훈련되어, 자연어 문장을 조건으로 다양한 이미지를 생성할 수 있다. 스케일(데이터 크기, 모델 크기)의 힘으로 VQ-GAN과 동일한 프레임워크에서 놀라운 **zero-shot 일반화** (훈련 시 본 적 없는 텍스트 설명에 대해서도 이미지를 생성할 수 있는 능력)를 달성했다.

6.2 DALL-E 2 (2022)

> 📄 **논문**: Ramesh et al., "Hierarchical Text-Conditional Image Generation with CLIP Latents", OpenAI, 2022

DALL-E 2는 방식이 크게 변화:

- 텍스트 임베딩: **CLIP** (Contrastive Language-Image Pre-training -- 텍스트와 이미지를 동일한 임베딩 공간에 매핑하여, 텍스트-이미지 유사도를 측정할 수 있게 한 모델) 사용 (DALL-E 1 시점에는 CLIP이 없었음)
- 이미지 생성: 자기회귀적 생성이 아닌 **Diffusion Model** (노이즈를 점진적으로 제거하여 이미지를 생성하는 확률적 모델) 사용
- 조건 부여: 단순 연결이 아닌 **인코더 + Cross-Attention** 방식

> 💡 **DALL-E 1 → DALL-E 2의 핵심 전환: 자기회귀 → 디퓨전** DALL-E 1의 자기회귀 방식은 256개 토큰을 **순차적으로** 생성해야 하므로 속도가 느리고, 이전 토큰의 오류가 이후 토큰에 누적되는 **에러 축적 (error accumulation)** 문제가 있었다. Diffusion Model은 모든 위치를 **동시에(parallel)** 생성하므로 속도가 빠르고, 반복적 정제(iterative refinement)를 통해 오류를 점진적으로 수정할 수 있다. 이 장점 때문에 DALL-E 2 이후 이미지 생성의 주류가 자기회귀에서 디퓨전으로 전환되었다.

---

7. 핵심 수식 총정리

> 이 섹션은 본문에서 다룬 모든 핵심 수식을 한눈에 볼 수 있도록 정리한 것이다. 각 수식의 상세한 유도와 직관적 설명은 해당 섹션을 참조하라.

CycleGAN

Adversarial Loss ($X \to Y$) -- "생성된 이미지가 목표 도메인의 실제 이미지와 구별 불가능하게":

$$\mathcal{L}_{\text{GAN}}(G_{X\to Y}, D_Y) = \mathbb{E}_y[\log D_Y(y)] + \mathbb{E}_x[\log(1 - D_Y(G_{X\to Y}(x)))]$$

Cycle Consistency Loss -- "변환 후 역변환하면 원본이 복원되어야 한다 (L1 거리)":

$$\mathcal{L}_{\text{cyc}} = \mathbb{E}_x[\|G_{Y\to X}(G_{X\to Y}(x)) - x\|_1] + \mathbb{E}_y[\|G_{X\to Y}(G_{Y\to X}(y)) - y\|_1]$$

전체 손실 -- "사실적 변환 + 구도 보존의 균형":

$$\mathcal{L} = \mathcal{L}_{\text{GAN}}(G_{X\to Y}, D_Y) + \mathcal{L}_{\text{GAN}}(G_{Y\to X}, D_X) + \lambda\mathcal{L}_{\text{cyc}}$$

핵심: λ가 두 목표의 균형을 결정한다. $\lambda = 10$이 기본값. 양방향 Adversarial Loss가 사실성을, Cycle Consistency Loss가 내용 보존을 담당한다.

StyleGAN AdaIN

$$\mathrm{AdaIN}(x, y) = y_s \cdot \frac{x - \mu(x)}{\sigma(x)} + y_b$$

핵심: 피처 맵 x의 통계량(평균, 분산)을 제거(정규화)한 뒤, 스타일 코드 w에서 유도된 새로운 통계량(y_s: 스케일, y_b: 바이어스)으로 교체한다. "기존 스타일 제거 → 새 스타일 주입"의 2단계 과정이다.

VQ-GAN

Stage 1 손실 -- "이미지를 이산 코드로 압축/복원 + 사실적 복원":

$$\mathcal{L}_{\text{Stage 1}} = \underbrace{\|x - \hat{x}\|^2 + \|sg[z_e] - e\|^2 + \beta\|z_e - sg[e]\|^2}_{\mathcal{L}_{\text{VQ}}} + \lambda\underbrace{(\mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(\hat{x}))])}_{\mathcal{L}_{\text{GAN}}}$$

$\mathcal{L}_{\text{VQ}}$의 세 항: Reconstruction(복원 품질) + Codebook(코드북이 인코더를 따라감) + Commitment(인코더가 코드북 근처에 머물러야 함). sg는 stop gradient(그래디언트 차단). $\mathcal{L}_{\text{GAN}}$은 복원 이미지의 사실성을 추가로 강제한다.

Stage 2 손실 (트랜스포머 디코더) -- "이전 토큰들이 주어졌을 때 다음 토큰의 확률 최대화":

$$\mathcal{L}_{\text{prior}} = -\sum_{i=1}^{N} \log p_\theta(s_i \mid s_{<i})$$

여기서 $p_\theta(s_i \mid s_{<i})$는 트랜스포머 디코더가 출력하는 소프트맥스 분포의 정답 위치 확률. 이것은 표준 Cross-Entropy Loss이며, GPT의 언어 모델링 손실과 완전히 동일하다.

자기회귀적 이미지 생성의 핵심 등식

$$p(\mathbf{s}) = \prod_{i=1}^{N} p(s_i \mid s_1, s_2, \ldots, s_{i-1})$$

이것은 **확률의 연쇄 법칙(chain rule of probability)**을 적용한 것이다: 결합 확률 $p(s_1, s_2, \ldots, s_N)$을 조건부 확률의 곱으로 분해한다. 이 분해는 항상 성립하며 (수학적 항등식), 핵심은 각 조건부 확률 $p(s_i \mid s_{<i})$를 **트랜스포머로 근사**한다는 것이다. 이것은 이미지 토큰 시퀀스의 결합 확률을 조건부 확률의 곱으로 분해한 것이며, **PixelRNN/CNN의 자기회귀적 분해와 동일한 원리**이다. 차이점은 픽셀이 아닌 **코드북 인덱스**를 대상으로 한다는 것.

---

8. 참고문헌

CycleGAN / Pix2Pix

1. **Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks** Zhu, J.-Y., Park, T., Isola, P., & Efros, A. A. (2017) ICCV 2017 https://arxiv.org/abs/1703.10593
2. **Image-to-Image Translation with Conditional Adversarial Networks (Pix2Pix)** Isola, P., Zhu, J.-Y., Zhou, T., & Efros, A. A. (2016) CVPR 2017 https://arxiv.org/abs/1611.07004

DiscoGAN

3. **Learning to Discover Cross-Domain Relations with Generative Adversarial Networks** Kim, T., Cha, M., Kim, H., Lee, J. K., & Kim, J. (2017) ICML 2017 https://arxiv.org/abs/1703.05192

StyleGAN Series

4. **A Style-Based Generator Architecture for Generative Adversarial Networks (StyleGAN v1)** Karras, T., Laine, S., & Aila, T. (2018) CVPR 2019 https://arxiv.org/abs/1812.04948
5. **Analyzing and Improving the Image Quality of StyleGAN (StyleGAN v2)** Karras, T., Laine, S., Aittala, M., et al. (2019) CVPR 2020 https://arxiv.org/abs/1912.04958
6. **Alias-Free Generative Adversarial Networks (StyleGAN v3)** Karras, T., Aittala, M., Laine, S., et al. (2021) NeurIPS 2021 https://arxiv.org/abs/2106.12423
7. **Progressive Growing of GANs for Improved Quality, Stability, and Variation (ProGAN)** Karras, T., Aila, T., Laine, S., & Lehtinen, J. (2017) ICLR 2018 https://arxiv.org/abs/1710.10196

VQ-GAN / VQ-VAE

8. **Taming Transformers for High-Resolution Image Synthesis (VQ-GAN)** Esser, P., Rombach, R., & Ommer, B. (2020) CVPR 2021 https://arxiv.org/abs/2012.09841
9. **Neural Discrete Representation Learning (VQ-VAE)** van den Oord, A., Vinyals, O., & Kavukcuoglu, K. (2017) NeurIPS 2017 https://arxiv.org/abs/1711.00937

DALL-E Series

10. **Zero-Shot Text-to-Image Generation (DALL-E)** Ramesh, A., Pavlov, M., Goh, G., et al. (2021) ICML 2021 https://arxiv.org/abs/2102.12092
11. **Hierarchical Text-Conditional Image Generation with CLIP Latents (DALL-E 2)** Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., & Chen, M. (2022) https://arxiv.org/abs/2204.06125

AdaIN / Instance Normalization

12. **Arbitrary Style Transfer in Real-time with Adaptive Instance Normalization** Huang, X. & Belongie, S. (2017) ICCV 2017 https://arxiv.org/abs/1703.06868